

计算机系统结构的基本概念



- ◆ 引言
- ◆ 计算机系统结构的定义
- ◆ 计算机系统的设计技术
- ◆ 计算机系统的评价标准
- ◆ 计算机系统结构的发展
- ◆ 计算机系统的分类



引 言

自第一台电子计算机问世已经半个多世纪了，它已经经历了五次更新换代：

- 第一代计算机（1945~1954）
- 第二代计算机（1955~1964）
- 第三代计算机（1965~1974）
- 第四代计算机（1975~1991）
- 第五代计算机（1992~现在）



第一代计算机

将**电子管**和**继电器存储器**用绝缘导线互连起来，**单个CPU**，**CPU**用程序计数器和累加器顺序完成定点运算，采用机器语言或汇编语言，**用CPU程序控制I/O**。代表性系统有：**John Von Neumann**、**Arthur Burks**和**Herman Goldstine**于**1946**年在普林斯顿大学研制成功的**IAS**计算机；宾夕法尼亚大学莫尔学院于**1950**年制成的**ENIAC**；**IBM**于**1953**年制造的**IBM701**计算机。



第二代计算机

采用分立式晶体三极管、二极管和铁氧体的磁芯，用印刷电路将它们互连起来。采用了变址寄存器、浮点运算、多路存储器和I/O处理机。采用有编译程序的高级语言、子程序库、批处理监控程序。代表性系统有：1959年制成的Univac LARC、60年代的CDC1604和1962年制成的IBM7030。



第三代计算机

采用**小规模或中规模集成电路**和**多层印刷电路**。微程序控制在这一代开始普及。采用了流水线、高速缓存和先行处理机。软件方面采用多道程序设计和分时操作系统。代表性系统有：**IBM 360/370系列、CDC 6600/7600系列、Texas仪表公司的ASC和Digital Equipment公司的PDP-8系列。**



第四代计算机

采用**大规模或超大规模集成电路和半导体存储器**，出现了用共享存储器、分布式存储器或向量硬件选择的不同结构的**并行计算机**，开发了用于并行处理的多处理操作系统、专用语言和编译器，同时产生了用于并行处理或分布处理的软件工具和环境。代表性系统有：**VAX9000、CrayX-MP、IBM 3090VF和BBNTC-2000等。**



第五代计算机

采用**VLSI**工艺更加完善的高密度、高速度处理机和存储器芯片。它最重要特点是进行大规模并行处理，采用可扩展的和容许时延的系统结构。代表性系统有：**Fujitsu**的**VPP500**、**Cray Research**的**MPP**、**Thinking Machines**公司的**CM-5**、**Intel**超级计算机系统**Paragon**、**SGI**的**Origin 2000**和**Sun**公司的**1000**服务器。



分 析

从计算机的发展过程，我们可以看出：
计算机系统性能的不断提高主要靠**器件的变革**和**系统结构的创新**。

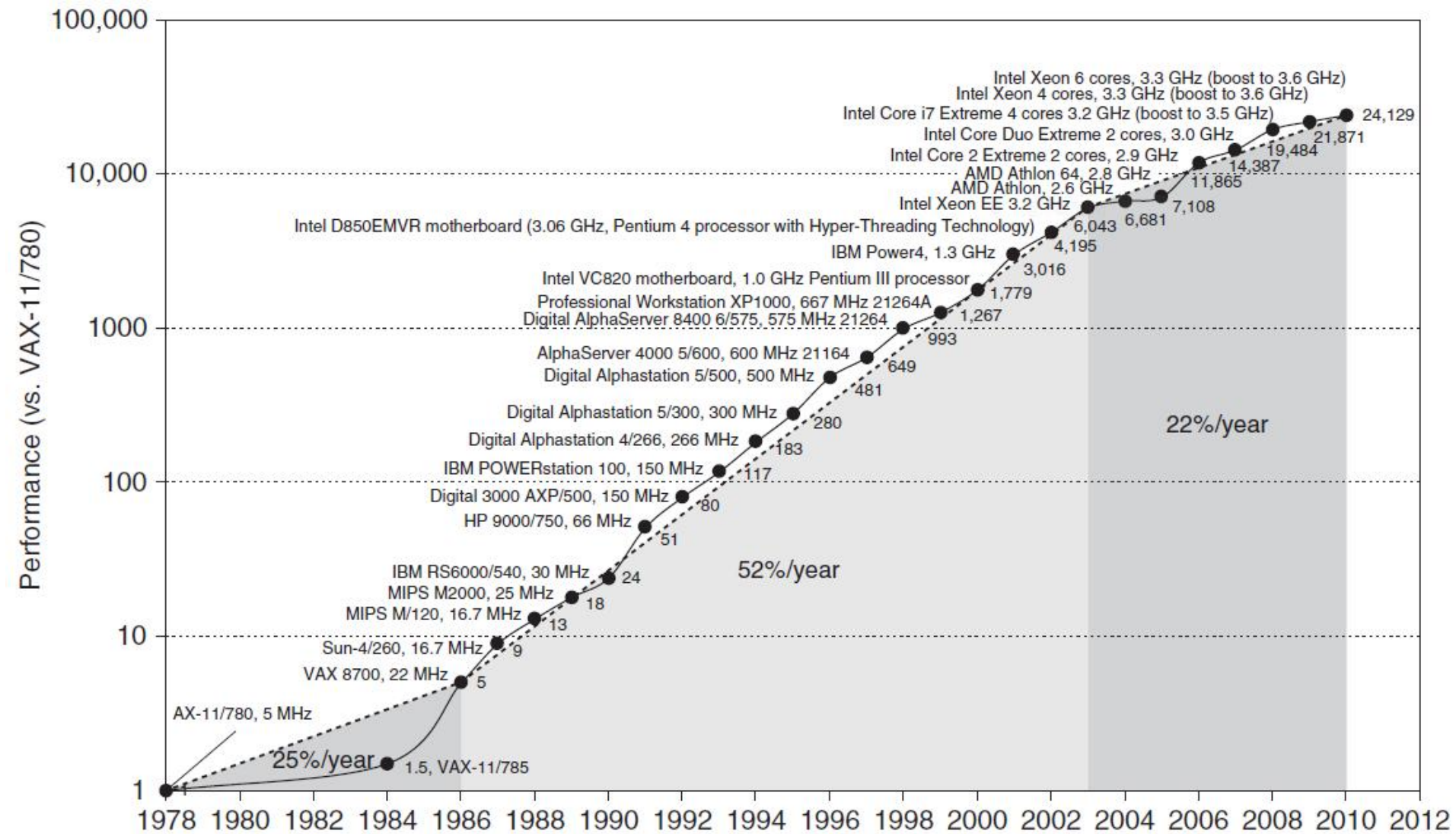
如何最合理地利用新器件，最大限度地发挥其潜力，设计并构成综合性能指标最佳的计算机系统，单纯依靠**器件变革**是不能解决的，还要靠**计算机系统结构**上的改进。



示 例

以**微处理器**为例（图示）。

- ◆ 在**20世纪80年代中期**以前，性能的提高主要是工艺技术驱动的，平均以每年**25%**的速度提高。
- ◆ 此后到**2002年**，性能的提高主要得益于工艺技术和先进的系统结构设计思想，该增长率达到了**52%**。
- ◆ **2002年后**，由于在**电源、可用的指令级并行和存储器长时延**等限制，使单处理器性能的提升减缓到**22%**。



通过SPEC基准测试程序，以VAX-11/780为基准的计算机性能





引出

本课程的目的：**采用定量的方法，并辅**以程序、实验和模拟等工具，来介绍计算机系统结构设计的常用方法和技术，系统结构的分析、设计和优化是本课程的重点。

让我们赶快开始吧！

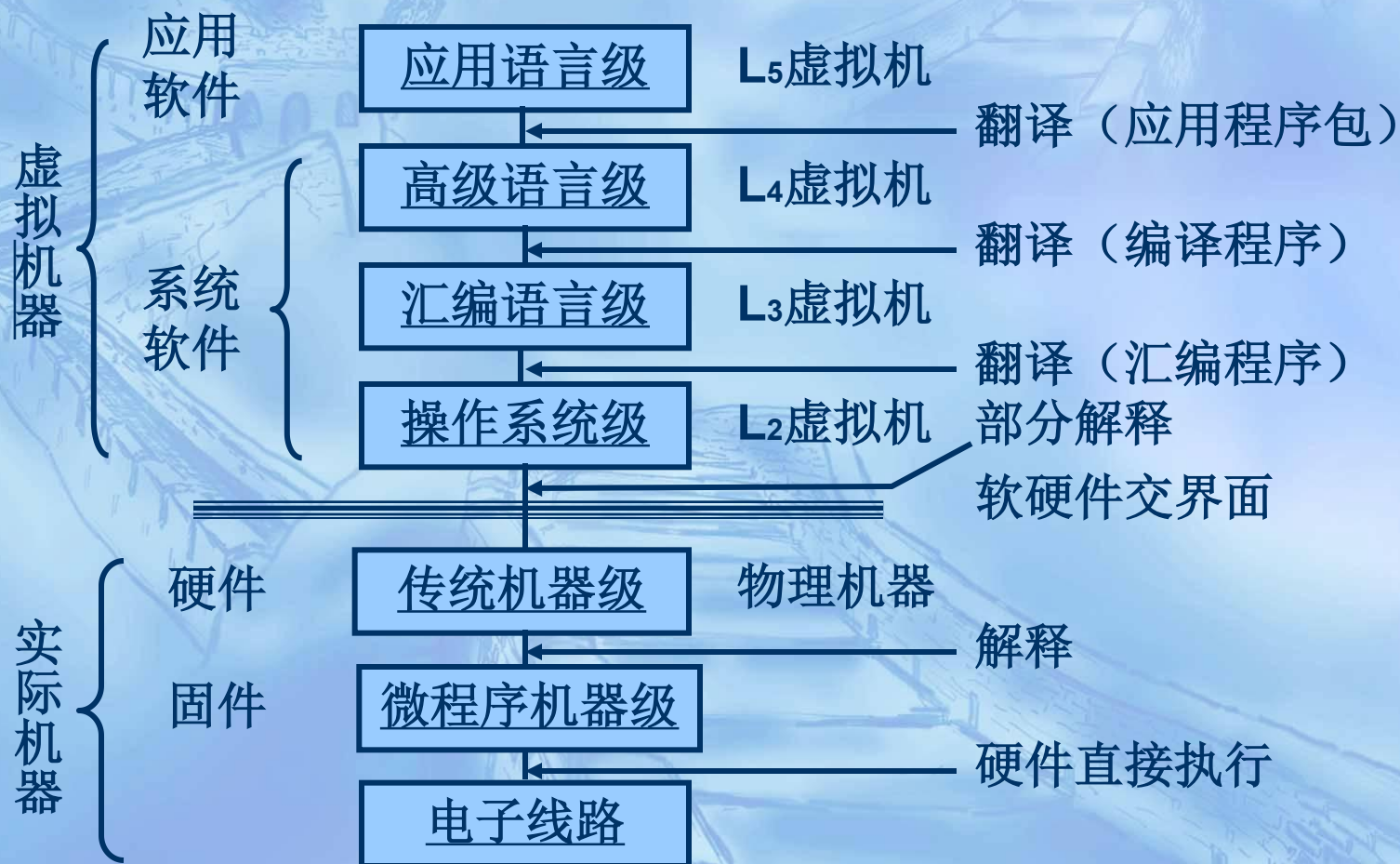


计算机系统结构 的定义

- 计算机系统的层次结构
- 计算机系统结构的概念
- 计算机组成的概念
- 计算机实现的概念



计算机系统层次结构图





应用语言级机器





高级语言级机器



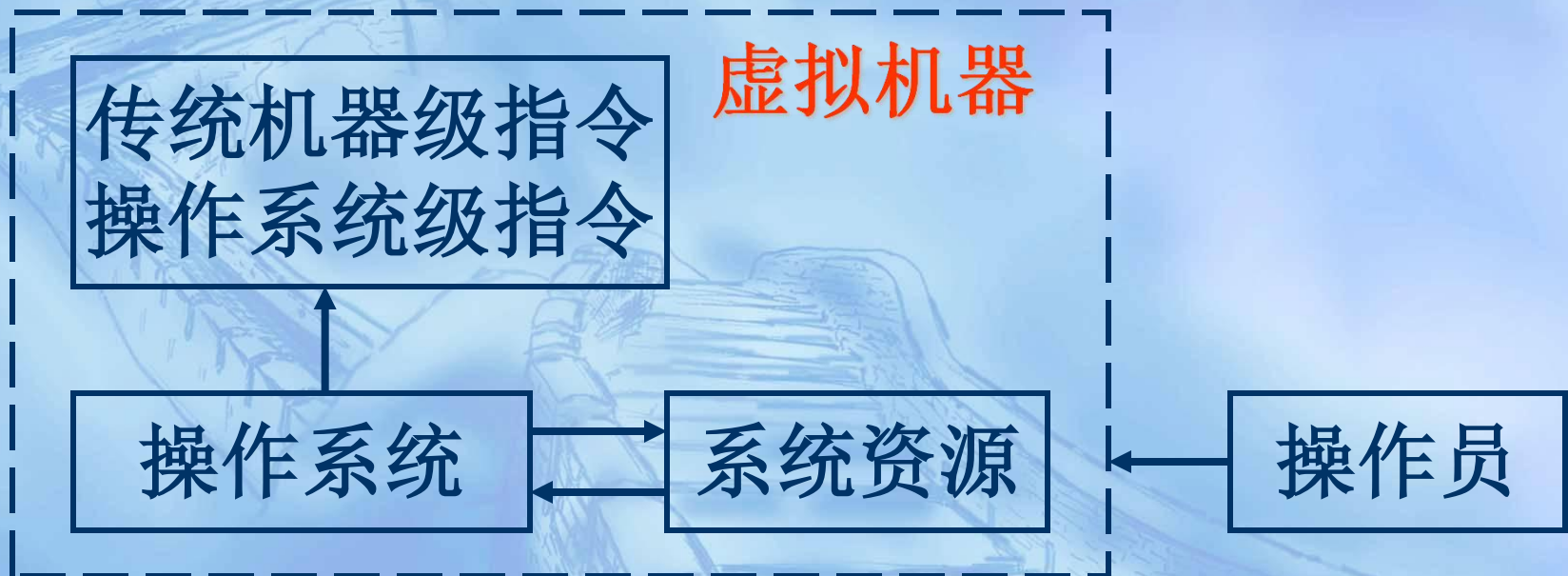


汇编语言级机器



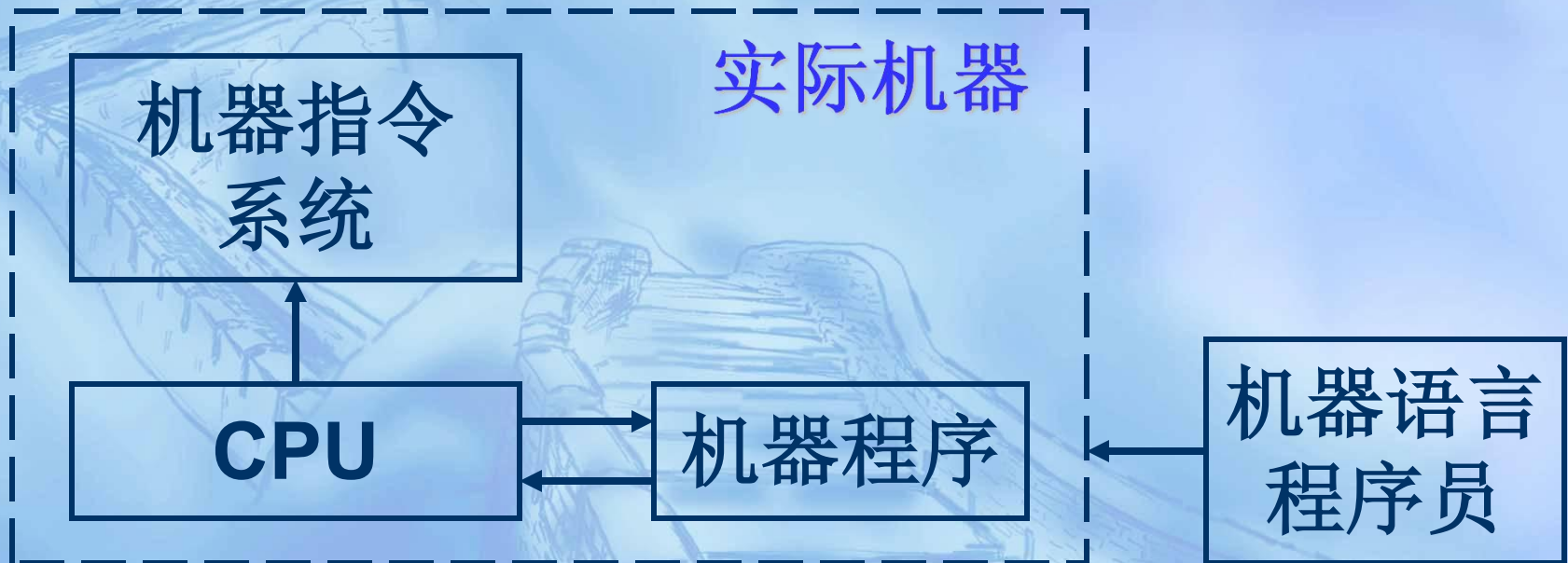


操作系统级机器



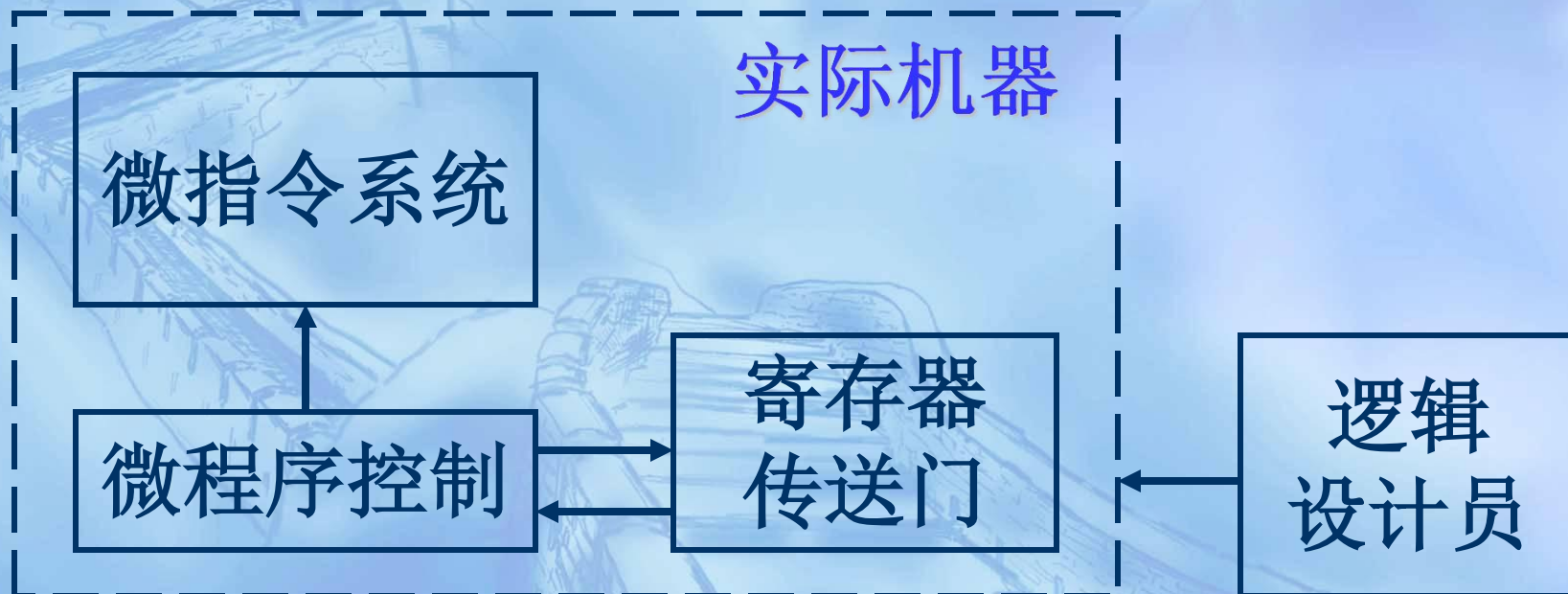


传统机器





微程序机器





电子线路





机器的作用和含义

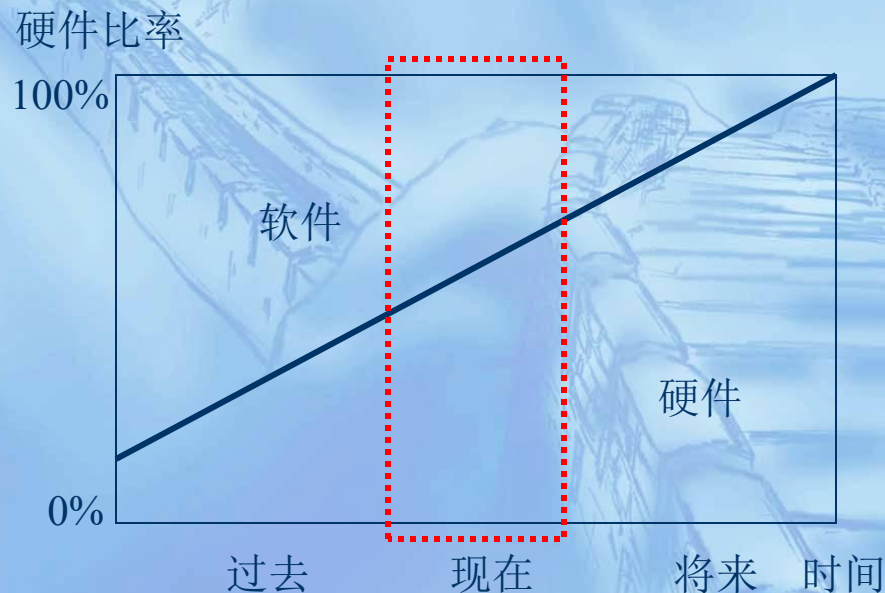
机器是指能执行和存储程序的算法和数据结构的集合体。**机器的实现**是指算法和数据结构的实现方法，可以硬件/固件/软件实现。我们将由软件实现的机器称为**虚拟机器**，将由硬件/固件实现的机器称为**实际机器**。

从计算机系统的某一层使用者的角度看，只需通过该层的语言就可以使用机器，而不必关心其下层的机器是如何工作和如何实现各自功能的，这一性质称为**透明性**。



软件与硬件的关系

- ❖ 系统是由硬件和软件组成的
- ❖ 软、硬件功能在逻辑上是等价的
- ❖ 软件和硬件的分界面是动态变化的



硬件比例高:

- 提高解题速度
- 减少存储量
- 硬件成本高
- 降低硬件利用率、系统灵活性和适应性

软件比例高:

- 降低硬件造价
- 提高系统灵活性和适应性
- 降低解题速度
- 增加系统存储量
- 增加软件设计费



分层的目的

- 有利于正确地理解计算机系统的工作，明确软件、硬件和固件在计算机系统中的地位和作用；
- 有利于理解各种语言的实质及其实现；
- 有利于探索虚拟机器新的实现方法，设计新的计算机系统。



计算机系统结构 的概念

“计算机系统结构”的含义仍有多种说法，并无统一的定义，下面介绍两种：

➤ 定义一

➤ 定义二



定义一

计算机系统结构是从程序员所看到的计算机的属性，即计算机的概念性结构和功能特性。（Amdahl, 1964）

哎！未指明程序员的级别

❖ 概念性结构

是指计算机系统中所有部件之间的逻辑连接结构。

❖ 功能特性

主要是指计算机系统的如下功能：数据表示、寻址方式、寄存器组织、指令系统、存储系统、中断机构、I/O结构等。



定义二

计算机系统结构是从机器语言程序员所看到的计算机的属性，即计算机的概念性结构和功能特性。即：计算机系统结构主要研究软硬件功能分配和对软硬件界面的确定。

明确多了！传统
机器级嘛！



主要任务

- **数据表示**: 指硬件能直接识别和处理的数据类型和格式等
- **寻址方式**: 包括最小寻址单位、寻址方式的种类、表示和地址计算等
- **寄存器组织**: 包括操作数寄存器、变址寄存器、控制寄存器和某些专用寄存器的定义、数量和使用约定
- **指令系统**: 包括机器指令的操作类型和格式, 指令间的排序方式和控制机构等
- **存储系统**: 包括最小编址单位、编址方式、主存容量、最大可编址空间等
- **中断机构**: 包括中断类型、中断分级、中断处理程序的功能和入口地址等
- **机器工作状态的定义和切换**
- **机器级的I/O结构**: 包括I/O的连接/访问方式、数据的源和目的、数据传送量、操作的结束和出错指示等
- **信息保护**: 包括保护方式、硬件对信息保护的支持



计算机组成的概念

- ❖ 计算机组成是计算机系统结构的逻辑实现，它包括传统机器级内的数据流和控制流的组织、逻辑设计等。
- ❖ 设计内容
 - 数据通路宽度
 - 控制机构的组成方式
 - 专用部件的设置
 - 缓冲和排队
 - 各种操作对部件的共享程度
 - 容错技术
 - 功能部件的并行度
 - 预测和评估
- ❖ 着眼于传统机器级内各事件的排序方式与控制方式、各部件的功能以及各部件之间的联系。



计算机实现的概念

- ◆ 计算机实现是计算机组成的物理实现
- ◆ 设计内容
 - 专用芯片（**ASIC**）的设计
 - 处理机、**Cache**和主存的物理结构
 - 器件、模块、插件和底板的逻辑划分和连接
 - 信号传输
 - 电源与冷却
 - 微组装和整机组装技术
 - **着眼于**器件技术和微组装技术，其中器件技术在其中起主导作用。



小 结

计算机系统结构研究计算机系统的软、硬件功能分配和软、硬件界面的确定；计算机组成是计算机系统结构的逻辑实现；计算机实现是计算机组成的物理实现。

具有相同系统结构的计算机可以因为速度等因素的要求而采用不同的组成；一种计算机的组成可以采用多种不同的计算机实现。

系统结构、组成和实现所包含的具体内容随不同时期及不同的计算机系统会有不同，而且随着技术、器件和应用的发展，三者之间的界限越来越模糊。



计算机系统的 设计技术

- ◆ 计算机系统设计者的任务
- ◆ 计算机系统的设计方法
- ◆ 计算机系统的设计步骤
- ◆ 计算机系统设计定量准则



计算机系统设计者 的任务

首先确定新计算机应具有哪些特性，然后在不超出成本的范围内力求性能最高。即：计算机设计者必须设计出一台既能满足功能要求，又能达到价格和性能目标的计算机。

主要任务有：

- 确定用户要求
- 优化设计
- 设计出符合今后发展方向的系统结构



确定用户要求

确定用户对计算机系统的**功能**、**价格**和**性能**的要求。

例如：具体的功能要求参见后表。

Functional requirements	Typical features required or supported
Application area	Target of computer
General purpose desktop	Balanced performance for a range of tasks, including interactive performance for graphics, video, and audio (Ch 2,3,4,5)
Scientific desktops and servers	High-performance floating point and graphics (App A,B)
Commercial servers	Support for databases and transaction processing, enhancements for reliability and availability. Support for scalability. (Ch 2,7)
Embedded computing	Often requires special support for graphics or video (or other application-specific extension). Power limitations and power control may be required. (Ch 2,3,4,5)
Level of software compatibility	Determines amount of existing software for machine
At programming language	Most flexible for designer; need new compiler (Ch 2,8)
Object code or binary compatible	Instruction set architecture is completely defined—little flexibility—but no investment needed in software or porting programs
Operating system requirements	Necessary features to support chosen OS (Ch 5,7)
Size of address space	Very important feature (Ch 5); may limit applications
Memory management	Required for modern OS; may be paged or segmented (Ch 5)
Protection	Different OS and application needs: page vs. segment protection (Ch 5)
Standards	Certain standards may be required by marketplace
Floating point	Format and arithmetic: IEEE 754 standard (App A), special arithmetic for graphics or signal processing
I/O bus	For I/O devices: Ultra ATA, Ultra SCSI, PCI (Ch 6)
Operating systems	UNIX, PalmOS, Windows, Windows NT, Windows CE, CISCO IOS
Networks	Support required for different networks: Ethernet, Infiniband (Ch 7)
Programming languages	Languages (ANSI C, C++, Java, Fortran) affect instruction set (Ch 2)





优化设计

最优设计方案的选择通常依赖于价格和性能（后面介绍）。设计者在处理价格和性能之间的关系时通常有三种不同的方法：

- 牺牲成本来换取高性能

例如：超级计算机（在2017.6月TOP500组织发布的最新一期世界超级计算机500强榜单中，神威·太湖之光超级计算机和天河二号超级计算机位居前两位）。

- 牺牲性能来换取低成本

例如：嵌入式系统。

- 性能价格比最优设计

例如：大多数的PC、工作站、服务器。



设计出符合今后发展方向的系统结构

一个成功的系统结构应该能经得住软、硬件技术的发展和应用的变化，因此设计者必须特别注意计算机应用和计算机技术的发展趋势，这样才能延长一种机器的使用寿命。



计算机系统的 设计方法

- ◆ 由上而下法
- ◆ 由下而上法
- ◆ 由中间向两头法



由上而下法

◆ 方法

- 确定应用的基本特性
- 设计或选择面向该应用的高级语言
- 设计适用于所选高级语言编译的中间语言
- 设计面向该应用的操作系统
- 设计面向所用编译器和操作系统的机器语言
- 设计面向机器语言的微指令及其硬件实现

◆ 评价

- 适用于面向某一应用的专用机的设计
- 设计周期较长（需若干年）
- 由于技术发展很快，因此难以真正面向用户优化实现



由下而上法

◆ 方法

- 不考虑应用，参照现有机器特点设计微程序机器级和传统机器级
- 选择面向不同应用的多种操作系统和编译器

◆ 评价

- 20世纪60~70年代较多采用的通用机的设计方法
- 因硬件不可更改，所以软件设计被动
- 系统的某些性能指标不确切，如传统机器级的“每秒运算次数”



由中间向两头法

◆ 方法

- 确定软、硬件界面
- 软件设计人员设计软件，硬件设计人员设计硬件
(同时)

◆ 评价

- 20世纪60年代末开始采用的通用机设计方法
- 设计周期短
- 交互式设计，便于提高系统性能
- 要求设计人员具有软件、硬件、器件和应用等方面的知识
- 需要完善的硬件系统模拟环境和软件设计环境



计算机系统的 设计步骤

◆ 需求分析

- 系统应用环境：实时处理、事务处理、科学计算、远程处理
- 所用语言的种类和特性
- 对操作系统的特殊要求
- 所用外设的特性
- 技术经济分析
- 市场分析

◆ 需求规格说明

- 设计准则
- 功能说明
- 器件性能说明

◆ 概要性设计

- 确定机器级界面

◆ 详细设计

- 定义机器级界面的内容，可提供多种方案

◆ 优化和评价

- 提高系统的性能价格比



计算机系统设计 的 定量准则

- ◆ 加快经常性事件的速度
- ◆ CPU性能公式
- ◆ 局部性原理
- ◆ 利用并行性

加快经常性事件的速度



在进行计算机系统设计时，必须优化（是指分配更多的资源、达到更高的性能或者分配更多的电能等）经常发生的功能，而非偶尔发生的功能。这是计算机系统设计中最重要也最广泛采用的设计准则。

例如：加法运算中的溢出和不溢出；中断处理中的常规性保护现场和特殊性保护现场。



Amdahl定律

计算机系统中某一部件由于采用某种更快的执行方式后整个系统性能的提高与这种执行方式的使用频率或占总执行时间的比例有关。

系统性能加速比:

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$



Amdahl定律 (量化)

$$S_n = \frac{T_0}{T_n} = \frac{1}{(1 - F_e) + \frac{F_e}{S_e}}$$

- ◆ S_n : 整个系统的加速比
- ◆ T_0 : 改进前整个任务的执行时间
- ◆ T_n : 改进后整个任务的执行时间
- ◆ F_e : 可改进比例

- ◆ S_e : 部件加速比

$$F_e = \frac{\text{可改进部分占用的时间}}{\text{改进前整个任务的执行时间}}$$
$$S_e = \frac{\text{改进前改进部分的执行时间}}{\text{改进后改进部分的执行时间}}$$



Amdahl定律

(例子)

问：假设某测试程序中**FP**指令执行时间占**50%**，**FPSQR**指令占**20%**，用改进**FPSQR**指令速度为原来的**10**倍和改进**FP**指令速度为原来的**2**倍，哪种方案更好？

答：方案一：**Fe=0.2, Se=10**，根据**Amdahl**定律，加速比为：

$$S_n = \frac{1}{(1 - 0.2) + \frac{0.2}{10}} = \frac{1}{0.82} = 1.22$$

方案二：**Fe=0.5, Se=2**，根据**Amdahl**定律，加速比为：

$$S_n = \frac{1}{(1 - 0.5) + \frac{0.5}{2}} = \frac{1}{0.75} = 1.33$$

结 论：方案二优于方案一



CPU性能公式

- ◆ 公式一
- ◆ 公式二
- ◆ 公式三



公式一

一个程序的**CPU**时间可以通过下式表达:

CPU时间 = 一个程序的CPU时钟周期数 × 时钟周期长度

或:

CPU时间 = $\frac{\text{一个程序的CPU时钟周期数}}{\text{时钟频率}}$

简单明了，没有什么实用价值



公式二

一个程序的CPU时间可以通过下式表达:

$$\text{CPU时间} = IC \times CPI \times \text{时钟周期长度}$$

其中: **IC**为一个程序的指令总数, **CPI**为执行一条指令所需的平均时钟周期数。

CPU时间与3个因素有关: 时钟周期长度、IC和CPI



公式三

一个程序的**CPU**时间可以通过下式表达:

$$\text{CPU时间} = \left(\sum_{i=1}^n \text{CPI}_i \times \text{IC}_i \right) \times \text{时钟周期长度}$$

其中: **IC_i**为指令*i*在一个程序中的执行次数, **n**为指令总数, **CPI_i**为执行指令*i*所需的平均时钟周期数。

$$\text{CPI} = \frac{\sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i)}{\text{IC}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}} \right)$$

CPU性能公式 (例子)



问：假设某测试程序中**FP**指令（包括**FPSQR**）的执行频度为**25%**，**FP**指令的平均**CPI=4.0**，其它指令的平均**CPI=1.33**；**FPSQR**指令的执行频度为**2%**，**FPSQR**指令的平均**CPI=20**。假设有两种设计方案：一种是将**FPSQR**的**CPI**减为**2**，另一种是将所有**FP**的**CPI**减为**2.5**，试利用**CPU**性能公式比较这两种设计方案。

答：原系统的**CPI**为：

$$CPI_{\text{原系统}} = \sum_{i=1}^n (CPI_i \times \frac{I_i}{IC}) = 4 \times 25\% + 1.33 \times 75\% = 2.0$$



CPU性能公式 (例子)

方案一：将FPSQR的CPI减为2

$$CPI_{\text{方案1}} = CPI_{\text{原系统}} - 2\% \times (CPI_{\text{老FPSQR}} - CPI_{\text{新FPSQR}}) = 2.0 - 2\% \times (20 - 2) = 1.64$$

方案二：将所有FP的CPI减为2.5

$$CPI_{\text{方案2}} = CPI_{\text{原系统}} - 25\% \times (CPI_{\text{老FP}} - CPI_{\text{新FP}}) = 2.0 - 25\% \times (4 - 2.5) = 1.625$$

或

$$CPI_{\text{方案2}} = \sum_{i=1}^n (CPI_i \times \frac{I_i}{IC}) = 1.33 \times 75\% + 2.5 \times 25\% = 1.625$$

结 论：方案二优于方案一



局部性原理

程序执行中呈现出频繁重复使用那些最近已使用过的数据和指令的规律。这反映在**时间局部性**和**空间局部性**上。

➤ 时间局部性

近期被访问的信息（数据和程序），可能马上被访问。

➤ 空间局部性

与被访问地址相邻的地址上的信息可能会一起被访问。

例：常用的一个经验规则

程序执行时间的90%都是在执行程序中10%的代码。



利用并行性

- ◆ 并行性的概念
- ◆ 并行性的实现
- ◆ 并行性的等级
- ◆ 并行性的发展
- ◆ 并行机的发展



并行性的概念

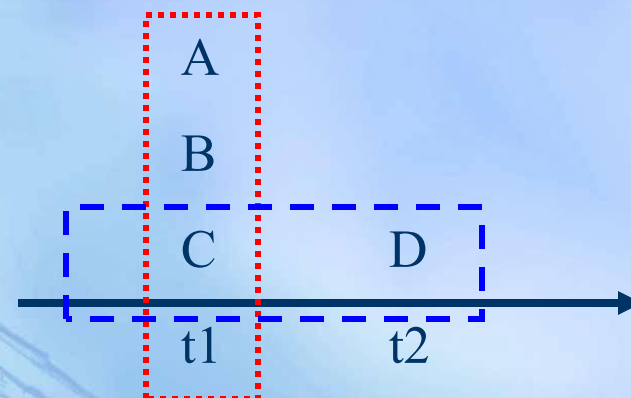
我们将问题中具有可以同时进行运算或操作的特性称为并行性。并行性实际上包含**同时性**和**并发性**两重含义：

➤ 同时性

是指两个或多个事件在同一时刻发生，例如：流水。

➤ 并发性

是指两个或多个事件在同一时间间隔内发生，例如：分时操作。





并行性的实现

- ◆ 时间重叠
- ◆ 资源重复
- ◆ 资源共享



时间重叠

➤ 思想

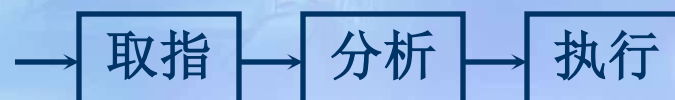
让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而提高速度。

➤ 例子

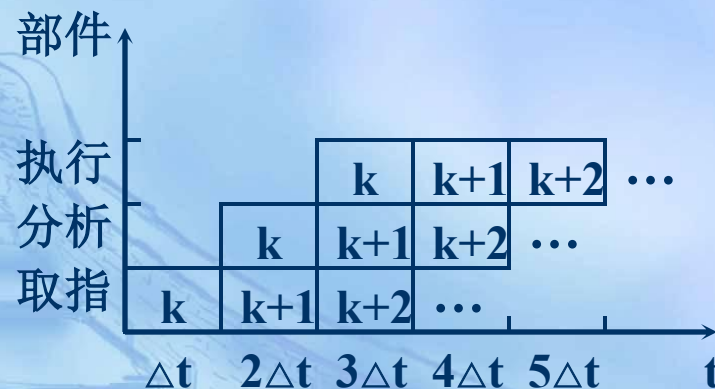
指令流水线。

➤ 特点

少量增加硬件设备就可以提高计算机系统的性能价格比。



(a) 指令流水线



(b) 时空图



资源重复

➤ 思想

通过重复设置资源（硬件、软件、信息、时间）来提高可靠性或性能。

➤ 例子

N模冗余结构→提高可靠性；多值存储器→提高信息存储密度；多处理机→提高速度和可靠性。

➤ 特点

需要增加资源才能大幅度提高计算机系统的性能。



资源共享

➤ 思想

利用软件的方法让多个用户按一定的时间顺序轮流地使用同一套资源，以提高其利用率，从而提高整个系统的性能。

➤ 例子

多道程序分时系统。

➤ 特点

不需要增加硬件设备就可以提高计算机系统的性能价格比。



并行性的等级

从不同的角度看，等级的划分也不一样：

- 从计算机系统中执行程序的角度看
- 从计算机系统中处理数据的角度看
- 从计算机信息加工的各个步骤和阶段看



从计算机系统中执行程序的角度看

- **指令内部并行**：一条指令内部各个微操作之间的并行。
- **指令级并行 (ILP)**：多条指令的并行执行。
- **线程级并行 (TLP)**：多个线程的并行执行，通常是以一个进程内派生的多个线程为调度单位。
- **任务级或进程级并行**：多个任务或进程的并行执行，以子程序或进程为调度单位。
- **作业级或程序级并行**：多个作业或程序的并行执行。



从计算机系统中处理数据的角度看

➤ 位串字符串

同时只对一个字的一位进行处理，这通常是指传统的串行单处理机，没有并行性。

➤ 位并字符串

同时对一个字的全部位进行处理，这通常是指传统的并行单处理机，开始出现并行性。

➤ 位片串字并

同时对许多字的同一位（位片）进行处理，开始进入并行处理领域。

➤ 全并行

同时对许多字的全部或部分进行处理。这是最高一级的并行。

位片

1011011

1101100

0100101

1001010

1110010

字



从计算机信息加工的 各个步骤和阶段看

- 存储器操作并行
例如，相联处理机。
- 处理器操作步骤并行
例如，流水线处理机。
- 处理器操作并行
例如，并行处理机。
- 指令、任务、作业并行
例如，多处理机。



并行性的发展

- ◆ 单处理机系统内并行性的发展
- ◆ 多处理机系统及其向并行处理系统的发展



单处理机系统内并行性的发展

◆ 时间重叠（主要）

把一个任务按功能分割为若干个相互联系的子任务,然后将每一个子任务分配给某个专门的部件处理。目的是使所有部件同时处于全面忙碌工作的状态。例如：指令流水线。

◆ 资源重复

例如：多体存储器、多操作部件、并行处理机等。

◆ 资源共享

例如：多道程序和分时系统



多处理机系统及其向并行处理系统的发展

◆ 时间重叠

多个处理机通过通道/通信线路实现互连，共享某些外设，以较低频带在文件/数据集级别上相互作用。系统功能被划分成多个专门功能，然后将它们分散给各个专用处理机。专用处理机可具有不同的体系结构。目标是构成异构型多处理机系统。

◆ 资源重复

通过总线/高速互连网络互连多个处理机，共享主存，以较高速率在数据/任务集上相互作用。支持进程和程序的并行处理。各处理机具有相同的功能。目标是构成同构型多处理机系统。

◆ 资源共享

地理上分散的多台计算机通过计算机通信网实现互连，共享资源。目标是构成分布式处理系统。



并行机的发展

1. 并行机的萌芽阶段（1964年～1975年）

- 20世纪60年代初期
 - **CDC6600**：非对称的共享存储结构，中央处理机采用了双**CPU**，并连接了多个外部处理器。
- 60年代后期，一个重要的突破
 - 在处理器中使用流水线和重复设置功能单元，所获得的性能提高是明显的，并比单纯地提高时钟频率来提高性能更有效。
- 在1972年，Illinois大学和Burroughs公司联合研制**Illiac IV SIMD**计算机（64个处理单元构成的）
 - 在1975年 **Illiac IV**系统（16个处理单元构成）



并行机的发展

2. 向量机的发展和鼎盛阶段（1976年～1990年）

- 1976年，Cray公司推出了第一台向量计算机Cray-1
- 在随后的10年中，不断地推出新的向量计算机。
 - 包括：CDC的Cyber205、Fujitsu的VP1000/VP2000、NEC的SX1/SX2以及我国的YH-1等
- 向量计算机的发展呈两大趋势
 - 提高单处理器的速度
 - 研制多处理器系统



并行机的发展

3. **MPP**出现和蓬勃发展阶段（1990年~1995年）

- 早期的**MPP**（大规模并行处理）
 - **TC2000**（1989年）、**Touchstone Delta**、**Intel Paragon**（1992年）、**KSR1**、**Cray T3D**（1993年）、**IBM SP2**（1994年）和我国的曙光-1000（1995年）等。（分布存储的**MIMD**计算机）
- **MPP**的高端机器
 - 1996年，**Intel**公司的**ASCI Red**和1997年**SGI Cray**公司的**T3E900**（万亿次高性能并行计算机）
- 90年代的中期，在中、低档市场上，**SMP**以其更优的性能/价格比代替了**MPP**。



并行机的发展

4. 各种体系结构并存阶段（1995年～2000年）

- 从1995年以后，**PVP**（并行向量处理机）、**MPP**、**SMP**、**DSM**（分布式共享存储多处理机）、**COW**等各种体系结构进入并存发展的阶段。
- **MPP**系统在全世界前**500**强最快的计算机中的占有量继续稳固上升，其性能也得到了进一步的提高。
 - 如：**ASCI Red**的理论峰值速度已达到了**1Tflop/s**
 - **SX4**和**VPP700**等的理论峰值速度也都达到了**1Tflop/s**



并行机的发展

5. 机群蓬勃发展阶段（2000年以后）

- 机群系统：将一群工作站或高档微机用某种结构的互连网络互连起来，充分利用其中各计算机的资源，统一调度、协调处理，以达到很高的峰值性能，并实现高效的并行计算。
- 1997年6月才有第一台机群结构的计算机进入Top500排名
- 2003年11月，这一数字已达到208台，机群首次成为Top500排名中比例最高的结构。
- 截至2008年6月，机群已经连续10期位居榜首，其数量已经达到400，占80%。



计算机系统的 评价标准

- ◆ 成本
- ◆ 性能



成本

◆ 成本

成本是指生产一个计算机系统所需的费用，包括软、硬件的费用。影响因素：**时间**、**产量**、**商品化**。

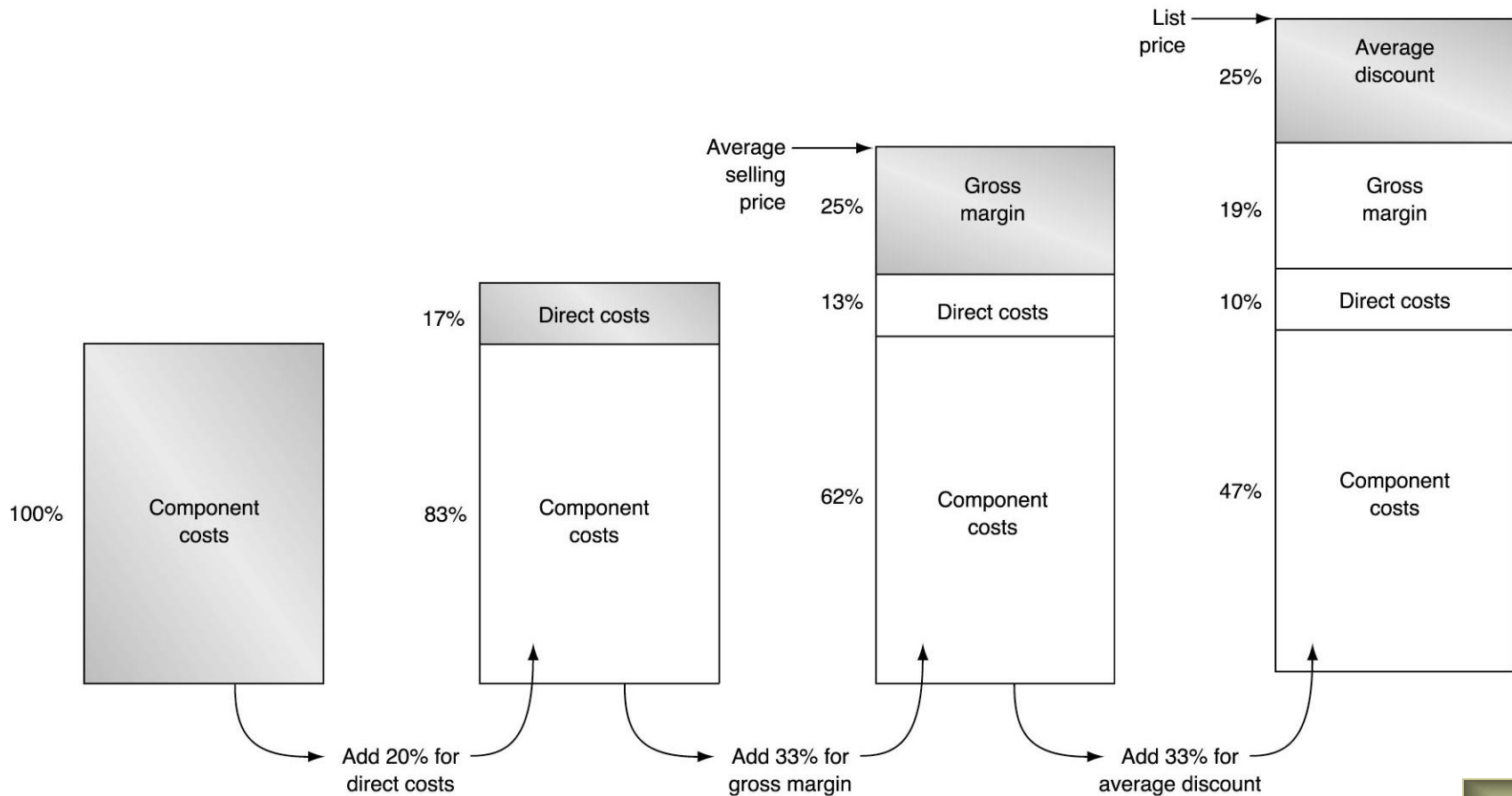
◆ 价格

价格是指一个计算机系统销售时的金额。以一个价值\$1000的PC为例（图）来介绍价格的构成：

标价 = 元件成本 + 直接成本 + 毛利 + 平均折扣

➤ **直接成本**是指与生产一件产品直接有关的成本，包括劳动力成本、保证金（在保质期间系统在用户那边失灵时的更换或维修费用）等。

➤ **毛利（间接成本）**是指与生产一件产品间接有关的成本，包括公司的研发费用、营销费用、设备维护费用、场地租金、财务成本、税前利润和税。





性能

- ◆ 性能标准
- ◆ 基准测试程序套件
- ◆ 性能比较



性能标准

衡量计算机性能的唯一永久而且可靠的标准是：**真实程序的执行时间**。下面介绍几种较流行的替代标准及其不足：

- ◆ MIPS
- ◆ MFLOPS
- ◆ 选择程序评价性能



MIPS

MIPS（每秒百万条指令数）定义为：

$$MIPS = \frac{\text{指令条数}}{\text{执行时间} \times 10^6} = \frac{\text{时钟频率}}{CPI \times 10^6}$$

程序的执行时间为：

$$T_e = \frac{\text{指令条数}}{MIPS \times 10^6}$$

遗憾啊！

主要缺点：

- 不同指令的速度差别很大
- 指令使用频度差别很大
- 有相当多的非功能性指令



MFLOPS

MFLOPS（每秒百万次浮点操作次数）定义为：

$$MFLOPS = \frac{\text{程序中的浮点操作次数}}{\text{执行时间} \times 10^6}$$

MFLOPS只能衡量机器浮点操作的性能，而不能体现机器的整体性能。

可借！

主要缺点：

- 依赖于浮点运算集
- 依赖于浮点操作类型



选择程序评价性能

主要有5类测试程序（以测量准确程度递减的次序排列）：

- 真实程序
- 改造/模拟程序
- 核心测试程序
- 小测试程序
- 合成测试程序



真实程序

➤ 思想

使用真实程序进行测试。

➤ 例子

C编译程序、**Word**等字处理程序、**Photoshop**等应用程序。

➤ 问题

会因程序依赖于具体的操作系统或编译器而产生移植问题。



改造/模拟程序

◆ 改造的目的

- 解决可移植问题
- 侧重于系统性能的某一特定方面

◆ 模拟的目的

能再现桌面系统中可能出现的相互影响的行为，或模拟服务器系统中可能出现的复杂的多用户交互。



核心测试程序

➤ 思想

从真实程序中提取出的一些小而关键的程序段进行测试。

➤ 例子

Livermore Loops和**Linpack**等。

➤ 用途

将计算机中各项特性的性能分离出来，以解释运行真实程序时性能有差异的原因。



小测试程序

➤ 思想

选择通常只有**10~100**行的代码、用户在测试之前都已经知道运行结果的程序进行测试。

➤ 例子

Puzzle、快速排序等。

➤ 用途

用于测试某种特定指标。



合成测试程序

➤ 思想

人工合成出来的测试程序。为了使测试程序的行为和表现尽可能接近于真实的应用程序，一般先对大量的应用程序中的操作进行统计，得到各种操作的比例，然后再按这个比例编制测试程序。

➤ 例子

Whetstone和Dhrystone。



基准测试程序套件

把应用程序中用得最频繁的那部分核心程序作为评价计算机性能的标准程序，称为**基准测试程序 (benchmark)**。将来自不同真实应用程序的基准测试程序组成**基准测试程序套件**，可以比较全面地反映计算机系统各方面的处理性能。

- ◆ 用于桌面机的benchmarks
- ◆ 用于服务器的benchmarks
- ◆ 用于嵌入式计算机的benchmarks



用于桌面机的 benchmarks

桌面机应具有的主要特性：**优化性能/价格比。**

SPEC (www.spec.org) 是常用的
benchmarks，这些benchmarks可分为两大类：
CPU和**图形**。

- 集中测试**CPU**的 benchmarks
SPEC CPU2000 (表, 测试实例)。
- 集中测试图形的 benchmarks
SPECviewperf和SPECapc。



SPEC

SPEC (System Performance Evaluation Cooperative) 是由30个左右世界知名计算机大厂商所支持的非盈利的合作组织，包括：IBM、AT&T、BULL、Compaq、CDC、DG、DEC、Fujitsu、HP、Intel、MIPS、Motolola、SGI、SUN、Unisys等；**SPEC能够全面反映机器的性能，具有很高的参考价值。**

SPEC发展过程为：

SPEC89→SPEC92→SPEC95→SPEC2000→SPEC2006

Benchmark	Type	Source	Description
gzip	Integer	C	Compression using the Lempel-Ziv algorithm
vpr	Integer	C	FPGA circuit placement and routing
gcc	Integer	C	Consists of the GNU C compiler generating optimized machine code.
mcf	Integer	C	Combinatorial optimization of public transit scheduling.
crafty	Integer	C	Chess playing program.
parser	Integer	C	Syntactic English language parser
eon	Integer	C++	Graphics visualization using probabilistic ray tracing
perlmbk	Integer	C	Perl (an interpreted string processing language) with four input scripts
gap	Integer	C	A group theory application package
vortex	Integer	C	An object-oriented database system
bzip2	Integer	C	A block sorting compression algorithm.
twolf	Integer	C	Timberwolf: a simulated annealing algorithm for VLSI place and route
wupwise	FP	F77	Lattice gauge theory model of quantum chromodynamics.
swim	FP	F77	Solves shallow water equations using finite difference equations.
mgrid	FP	F77	Multigrid solver over 3-dimensional field.
apply	FP	F77	Parabolic and elliptic partial differential equation solver
mesa	FP	C	Three dimensional graphics library.
galgel	FP	F90	Computational fluid dynamics.
art	FP	C	Image recognition of a thermal image using neural networks
equake	FP	C	Simulation of seismic wave propagation.
facerec	FP	C	Face recognition using wavelets and graph matching.
ammp	FP	C	molecular dynamics simulation of a protein in water
lucas	FP	F90	Performs primality testing for Mersenne primes
fma3d	FP	F90	Finite element modeling of crash simulation
sixtrack	FP	F77	High energy physics accelerator design simulation.
apsi	FP	F77	A meteorological simulation of pollution distribution.

12个整数程序用于
CINT2000测试

14个浮点程序用于
CFP2000测试





SPEC CPU2000

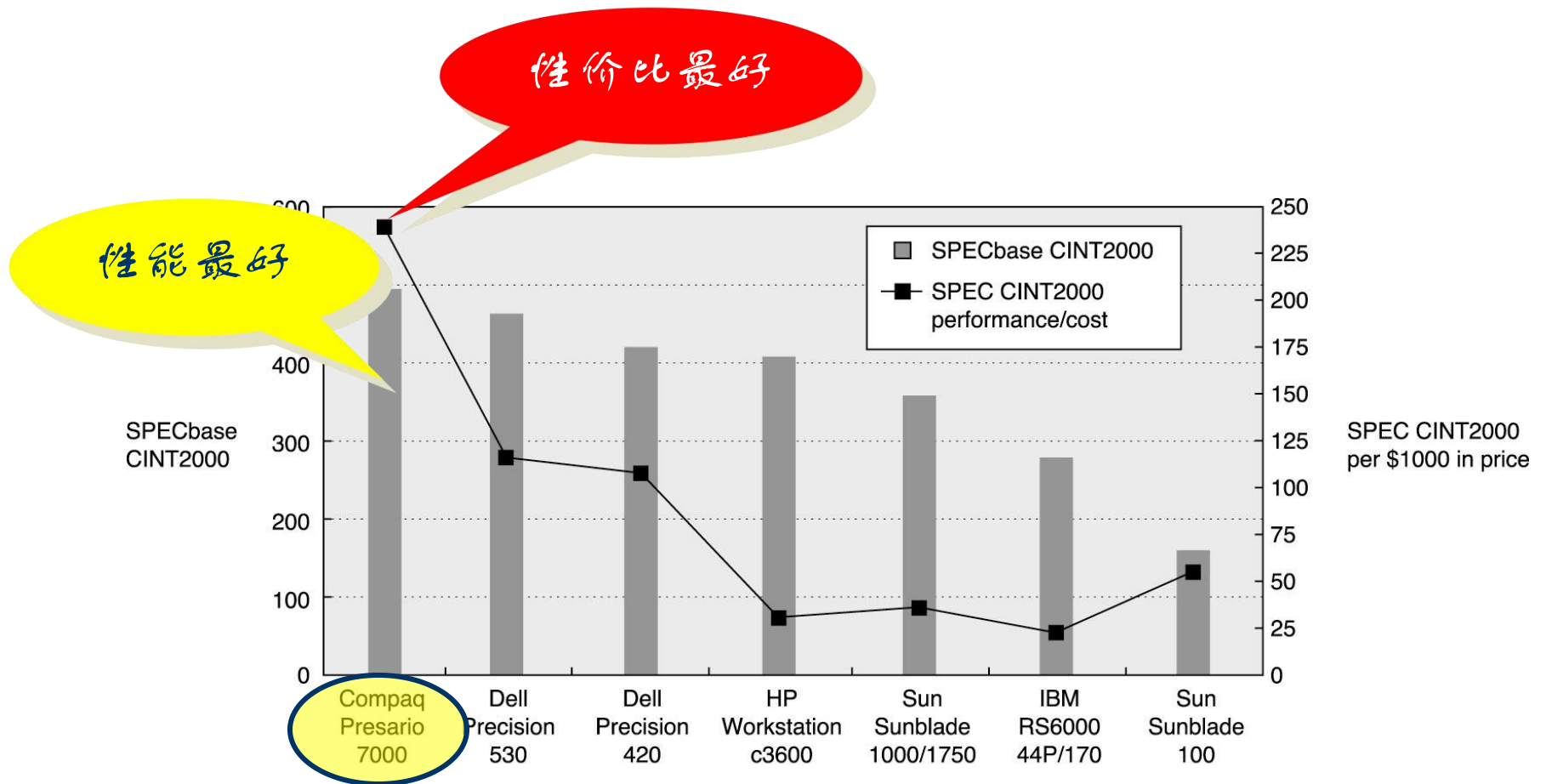
测试实例

- ◆ 测试对象
- ◆ 测试结果
 - SPEC CINT2000的测试结果
 - SPEC CFP2000的测试结果

Vendor	Model	Processor	Clock Rate (MHz)	Price
Compaq	Presario 7000	AMD Athlon	1,400	\$2,091
Dell	Precision 420	Intel Pentium III	1,000	\$3,834
Dell	Precision 530	Intel Pentium 4	1,700	\$4,175
HP	Workstation c3600	PA 8600	552	\$12,631
IBM	RS6000 44P/170	IBM III-2	450	\$13,889
Sun	Sunblade 100	UltraSPARC II-e	500	\$2,950
Sun	Sunblade 1000	UltraSPARC III	750	\$9,950

每一个桌面系统都配有一个**CPU**、**512MB SDRAM**（如果支持则带**ECC**校验）、大约**20GB**的硬盘、一个快速的图形系统和一个**10/100Mbps**以太网连接。



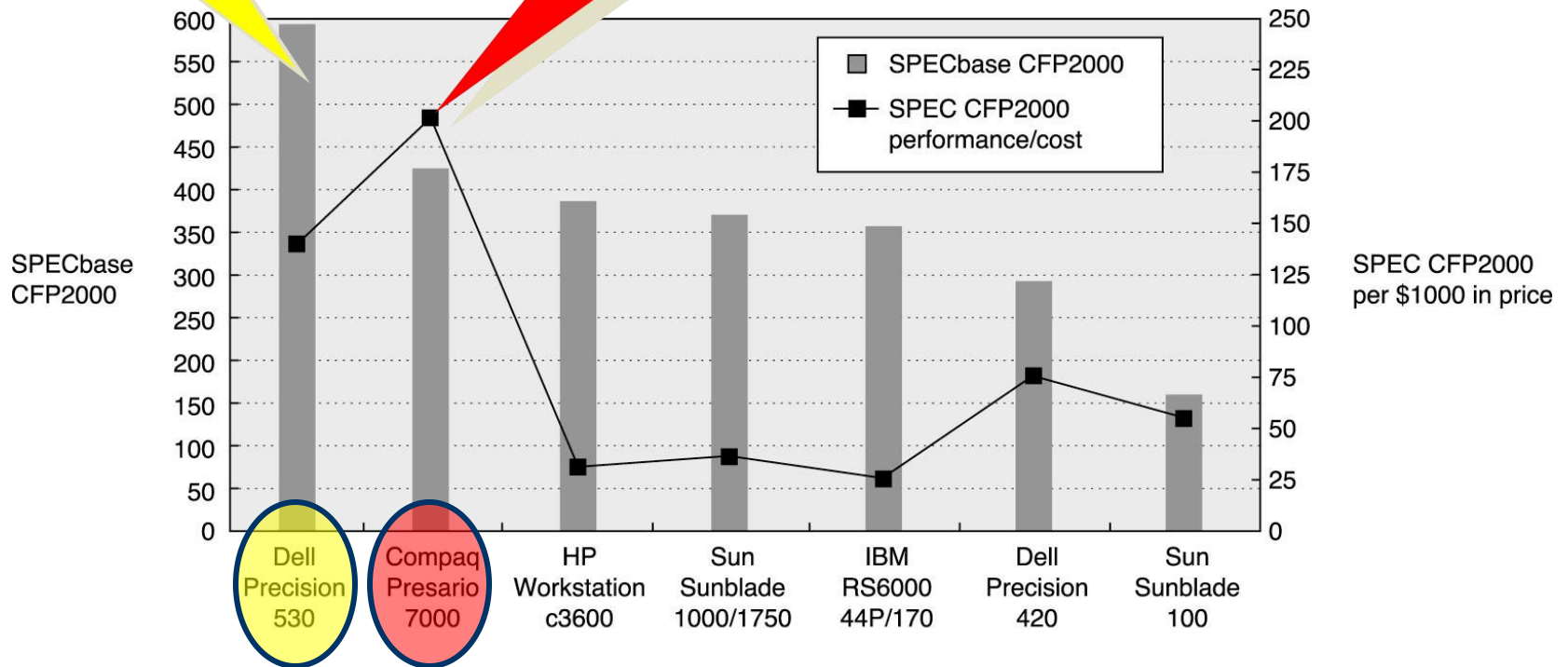


7个桌面计算机的SPEC CINT2000测试结果



性能最好

性价比最好



7个桌面计算机的SPEC CFP2000测试结果





用于服务器的 benchmarks

服务器应具有的主要特性：优化**可用性**
（**availability**）、**可扩展性**（**scalability**）
和吞吐量（**throughput**）。

常用的**benchmarks** 有两种：

- **SPEC**
- **TPC**（Transaction Processing Council）



SPEC

➤ SPECrate

面向流量的benchmarks。

➤ SPECIFS

面向文件服务器(NFS)的benchmarks。它不仅测试处理器的性能，而且测试I/O系统的性能。它重点测试吞吐率。

➤ SPECWeb

面向Web服务器的benchmarks。它模拟多个客户请求访问服务器中的动态和静态页面以及向服务器上传数据的情况。



TPC

TPC的benchmarks (www.tpc.org) 用于测量一个计算机系统事务处理的性能。第一个**TPC benchmarks**是**TPC-A**，现已被下面的几个所取代：

➤ **TPC-C**

TPC-C于1992建立，用于模拟一个复杂的查询环境。

➤ **TPC-H**

TPC-H用于自主（**ad hoc**）决策支持。

➤ **TPC-W**

TPC-W是一个基于**Web**的事务**benchmark**。

➤ **TPC-E**

TPC-E用于模拟一个经纪公司的客户账户。



TPC-C测试实例

- ◆ 测试对象
- ◆ 测试结果
 - 6个高性能服务器的测试结果
 - 6个高性价比服务器的测试结果

Vendor & System	CPUs	Database	OS	Price
IBM exSeries 370 c/s	280 x Pentium III @ 900 Mhz	Microsoft SQL Server 2000	Microsoft Windows Adv. Server	\$15,543,346
Compaq Alpha server GS 320	32 x Alpha 21264 @ 1GHz	Oracle 9i	Compaq Tru64 UNIX	\$10,286,029
Fujitsu PRIMEPOW-ER 20000	48 x SPARC64 GP @ 563 MHz	SymfoWARE Server Enterpr.	Sun Solaris 8	\$9,671,742
IBM eServer 680 7017-S85	24 x IBM RS64-IV 600 MHz	Oracle 8 8.1.7.1	IBM AIX 4.3.3	\$7,546,837
HP 9000 Enterprise Server	48 x HP PA-RISC 8600 552 MHz	Oracle8 v8.1.7.1	HP UX 11.i 64-bit	\$8,522,104
IBM eServer 400 840-2420	24 x iSeries400 Model 840	IBM DB2 for AS/400 V4	IBM OS/400 V4	\$8,448,137
Dell PowerEdge 6400	3 x Pentium III 700MHz	Microsoft SQL Server 2000	Microsoft Windows 2000	\$131,275
IBM eserver xSeries 250 c/s	4 x Pentium III 700 MHz	Microsoft SQL Server 2000	Microsoft Windows Adv. Server	\$297,277
Compaq Proliant ML570 6/700 2	4 x Intel Pentium III @ 700 MHz	Microsoft SQL Server 2000	Microsoft Windows Adv. Server	\$375,016
HP NetServer LH 6000	6 x Pentium III @ 550 MHz	Microsoft SQL Server 2000	Microsoft Windows NT Enterprise	\$372805
NEC Express 5800/180	8 x Pentium III 900 MHz	Microsoft SQL Server 2000	Microsoft Windows Adv. Server	\$682,724
HP 9000 / L2000	4 x PA-RISC 8500 440MHz	Sybase Adaptive Server	HP UX 11.0 64-bit	\$368,367

高性能

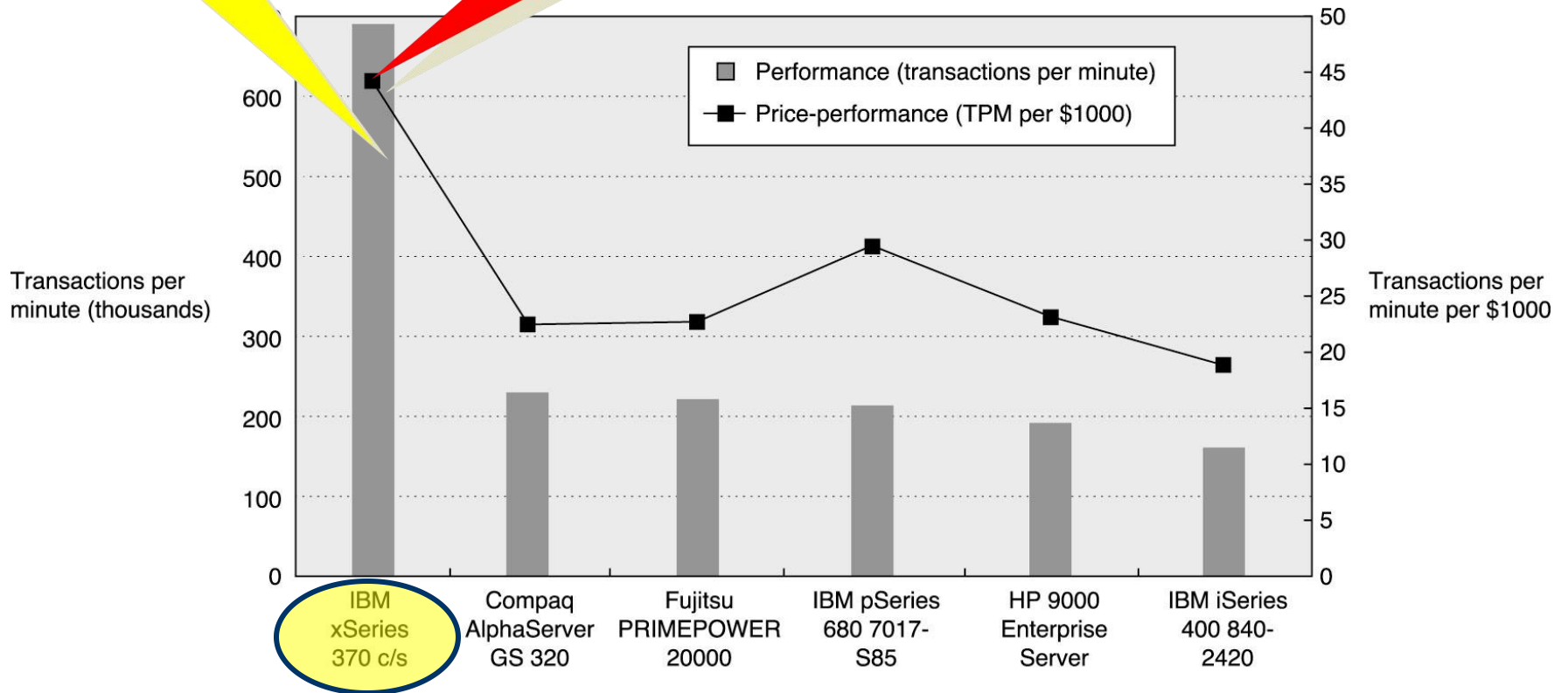
高性价比

12个OLTP（在线事务处理）服务器



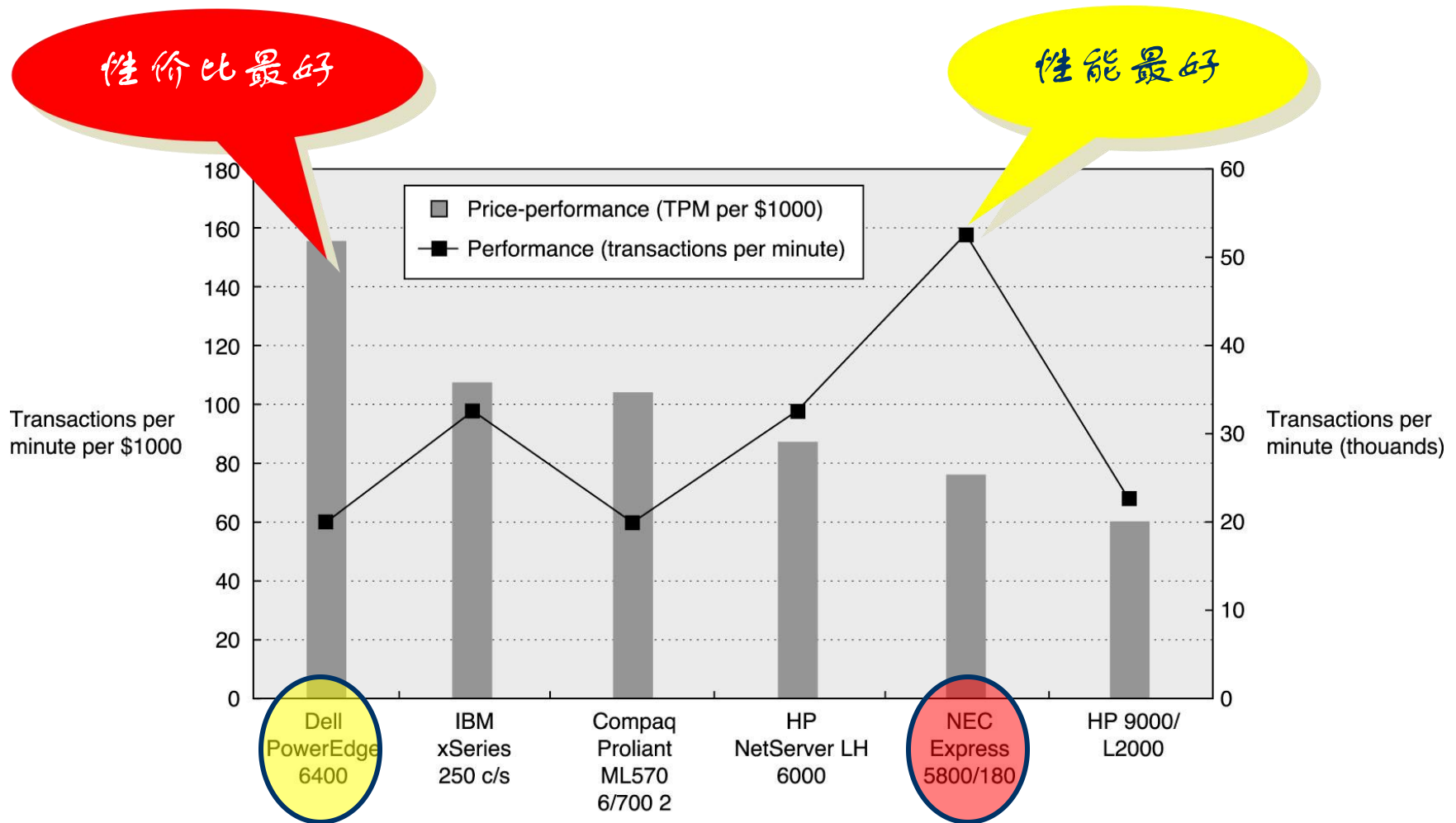
性能最好

性价比最好



6个高性能OLTP服务器的TPC-C测试结果





6个高性价比OLTP服务器的TPC-C测试结果





用于嵌入式计算机的 benchmarks

嵌入式计算机应具有的主要特性：优化**软件、价格、实时性、内存和功率**。

benchmark的发展处于初期阶段，目前最好的benchmark标准化集是**EEMBC**（EDN Embedded Microprocessor Benchmark Consortium）。EEMBC benchmarks可分为5类：汽车/工业、消费者、网络、办公自动化和电信（表，测试实例）。

Benchmark Type	# of this type	Example benchmarks
Automotive/industrial	16	6 microbenchmarks (arithmetic operations, pointer chasing, memory performance, matrix arithmetic, table lookup, bit manipulation), 5 automobile control benchmarks, and 5 filter or FFT benchmarks.
Consumer	5	5 multimedia benchmarks (JPEG compress/decompress, filtering, and RGB conversions).
Networking	3	Shortest path calculation, IP routing, and packet flow operations.
Office automation	4	Graphics and text benchmarks (Bezier curve calculation, dithering, image rotation, text processing).
Telecommunications	6	Filtering and DSP benchmarks (autocorrelation, FFT, decoder, and encoder)

EEMBC benchmarks (www.eembc.org)





EEMBC测试实例

- ◆ 测试对象

- ◆ 测试结果

- 相对性能的测试结果

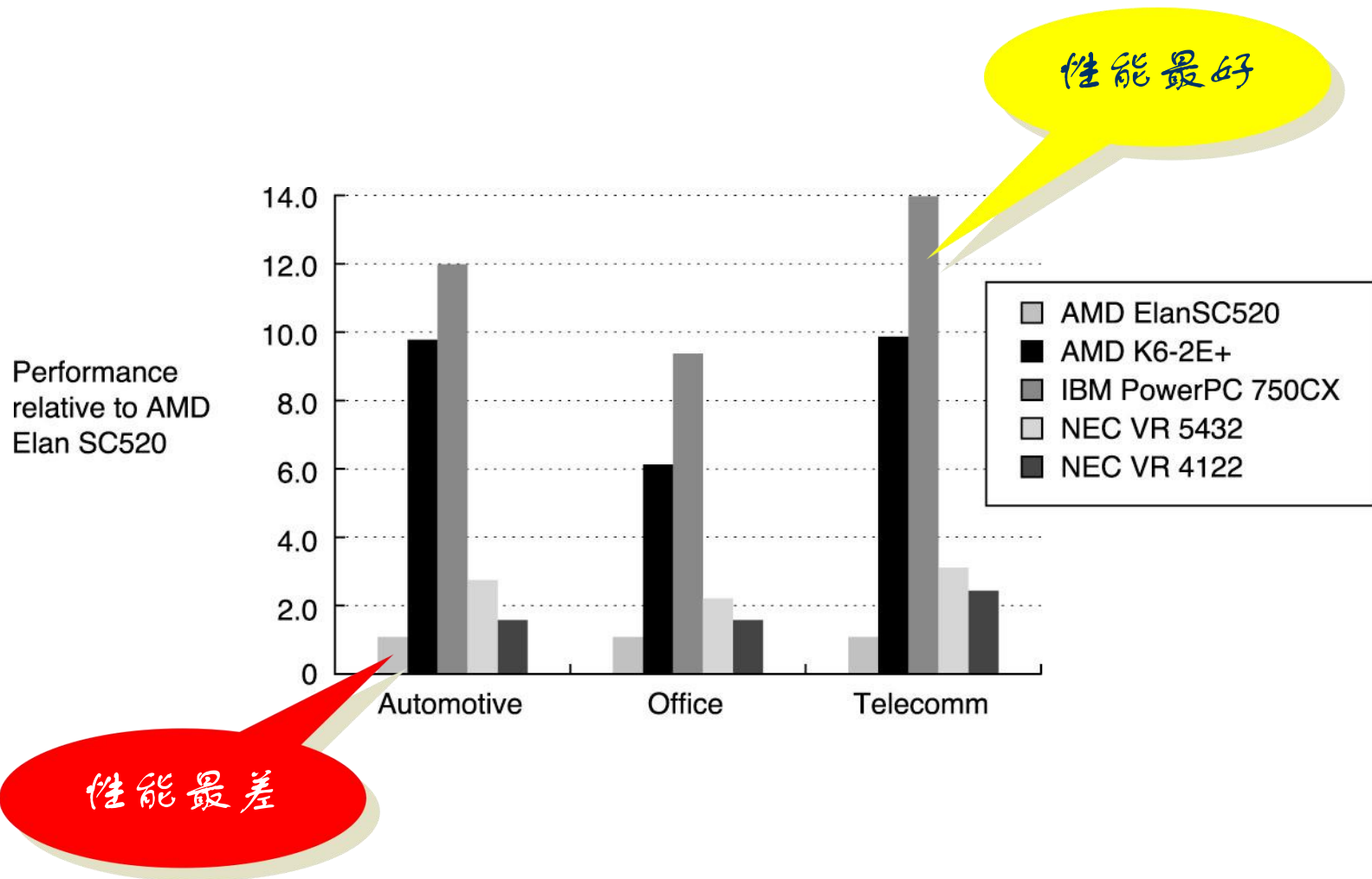
- 相对性价比的测试结果

- 功率和效率的测试结果

Processor	Instr. Set	Processor Clock Rate (MHz)	Cache Instr./Data On-chip Secondary cache	Processor organization	Typical power (in mW)	Price (\$)
AMD Elan SC520	x86	133	16K/16K	Pipelined: single issue	1600	\$38
AMD K6-2E+	x86	500	32K/32K 128K	Pipelined: 3+ issues/clock.	9600	\$78
IBM PowerPC 750CX	PowerPC	500	32K/32K 128K	Pipelined 4 issues/clock	6000	\$94
NEC VR 5432	MIPS-64	167	32K/32K	Pipelined: 2 issues/clock	2088	\$25
NEC VR 4122	MIPS-64	180	32K/16K	Pipelined: single issue	700	\$33

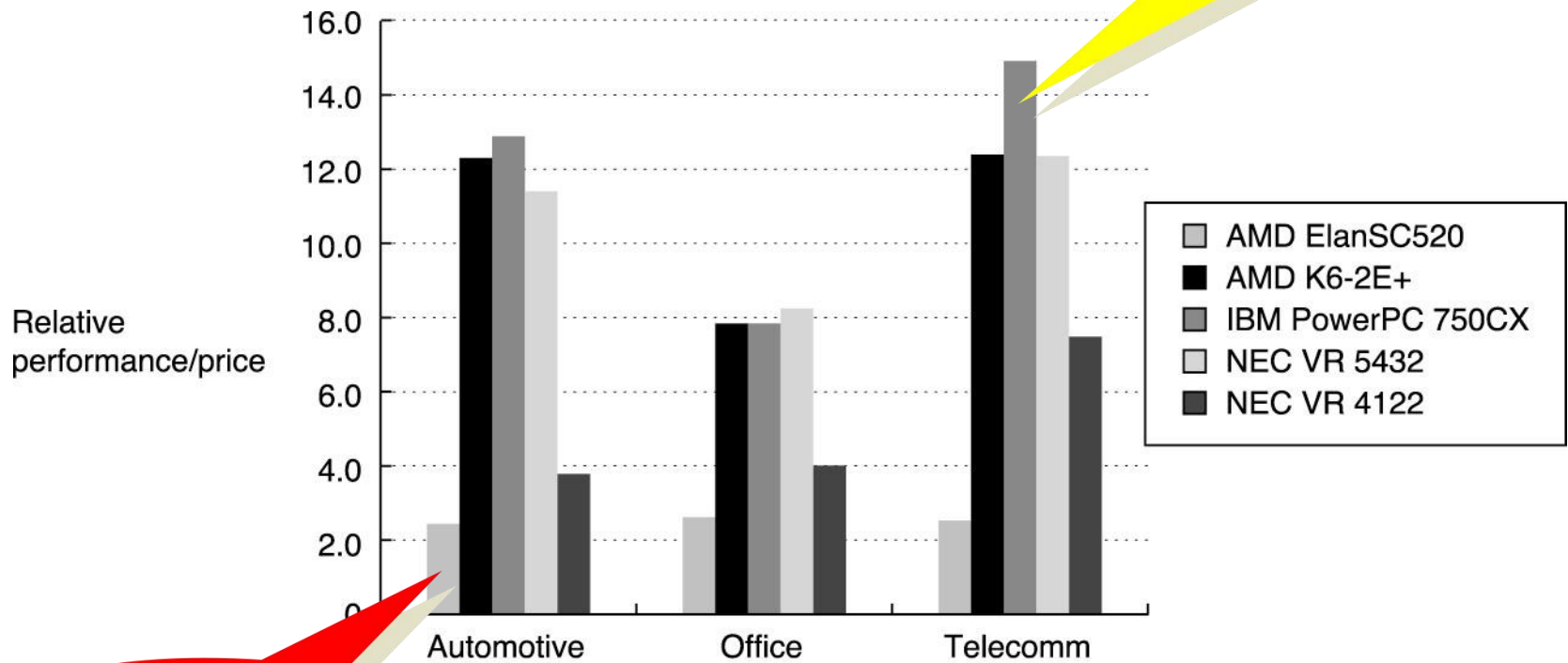
5个不同的嵌入式处理器





5个嵌入式处理器相对性能的EEMBC测试结果



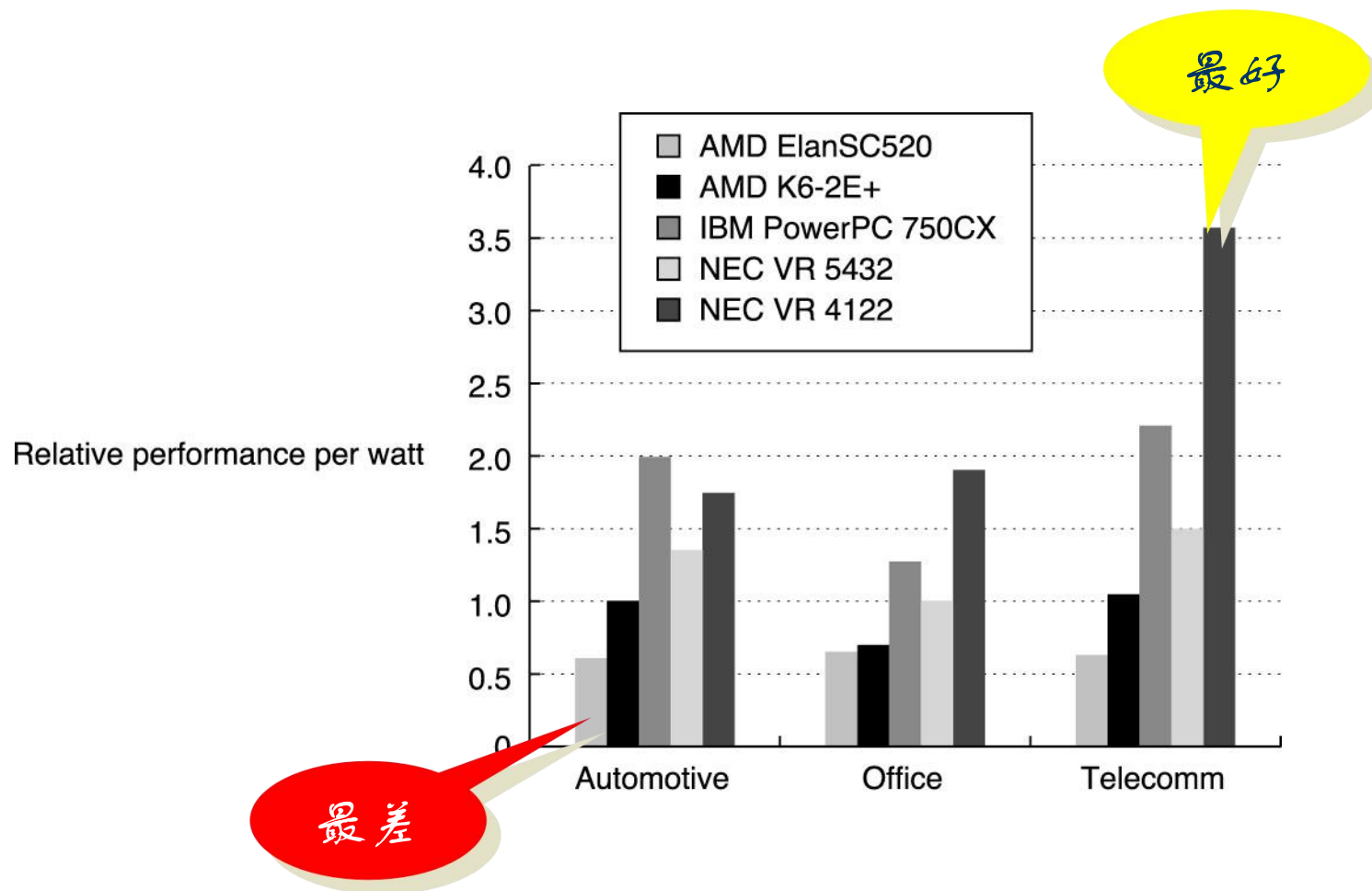


性价比最差

性价比最好

5个嵌入式处理器相对性价比的EEMBC测试结果





5个嵌入式处理器每瓦相对性能的EEMBC测试结果





性能比较

为了能公平地综合比较计算机的性能，
引入新的综合度量标准：

- ◆ 各程序的比重相同时
 - 总执行时间
- ◆ 各程序的比重不同时
 - 加权执行时间
 - 标准化执行时间



总执行时间

总执行时间

平均执行时间(时间)
算术平均

平均执行时间(速度)
调和平均

$$S_m = \sum_{i=1}^n T_i$$

$$A_m = \frac{1}{n} \sum_{i=1}^n T_i$$

$$H_m = \frac{n}{\sum_{i=1}^n \frac{1}{R_i}}$$

其中: T_i 为第 i 个程序的执行时间, n 为程序数, $R_i = 1/T_i$ 。

	Computer A	Computer B	Computer C
Program P1 (secs)	1	10	20
Program P2 (secs)	1000	100	20
Total time (secs)	1001	110	40



加权执行时间

加权算术平均: 加权调和平均:

$$A_m = \sum_{i=1}^n W_i \times T_i$$

$$H_m = \left(\sum_{i=1}^n \frac{W_i}{R_i} \right)^{-1}$$

W_i : 为第*i*个程序的比重

T_i : 为第*i*个程序的执行时间

R_i : $1/T_i$

n : 为程序数

	Programs			Weightings		
	A	B	C	W(1)	W(2)	W(3)
Program P1 (secs)	1.00	10.00	20.00	0.50	0.909	0.999
Program P2 (secs)	1000.00	100.00	20.00	0.50	0.091	0.001

Arithmetic mean:W(1)	500.50	55.00	20.00
Arithmetic mean:W(2)	91.91	18.19	20.00
Arithmetic mean:W(3)	2.00	10.09	20.00



标准化执行时间

先将各程序的执行时间对一台参考机器进行**标准化**，然后取标准化执行时间的**平均值**（可以是**算术平均值**，也可以是**几何平均值**）。

算术平均：

$$A_m = \frac{1}{n} \sum_{i=1}^n ETR_i$$

几何平均：

$$G_m = \sqrt[n]{\prod_{i=1}^n ETR_i}$$

ETR_i：为第i个程序对参考机器标准后的执行时间

n：为程序数

这是**SPEC**基准测试程序所用的方法，它的参考时间是以在**SPARCstation**上的运行时间作为基准。



标准化执行时间

	Computer A	Computer B	Computer C
Program P1 (secs)	1	10	20
Program P2 (secs)	1000	100	20
Total time (secs)	1001	110	40

	Normalized to A			Normalized to B			Normalized to C		
	A	B	C	A	B	C	A	B	C
Program P1	1.0	10.0	20.0	0.1	1.0	2.0	0.05	0.5	1.0
Program P2	1.0	0.1	0.02	10.0	1.0	0.2	50.0	5.0	1.0
Arithmetic mean	1.0	5.05	10.01	5.05	1.0	1.1	25.03	2.75	1.0
Geometric mean	1.0	1.0	0.63	1.0	1.0	0.63	1.58	1.58	1.0
Total time	1.0	0.11	0.04	9.1	1.0	0.36	25.03	2.75	1.0

算术平均值因参考机器不同而不同，几何平均值不因参考机器不同而不同。所以标准化执行时间应采用几何平均值。



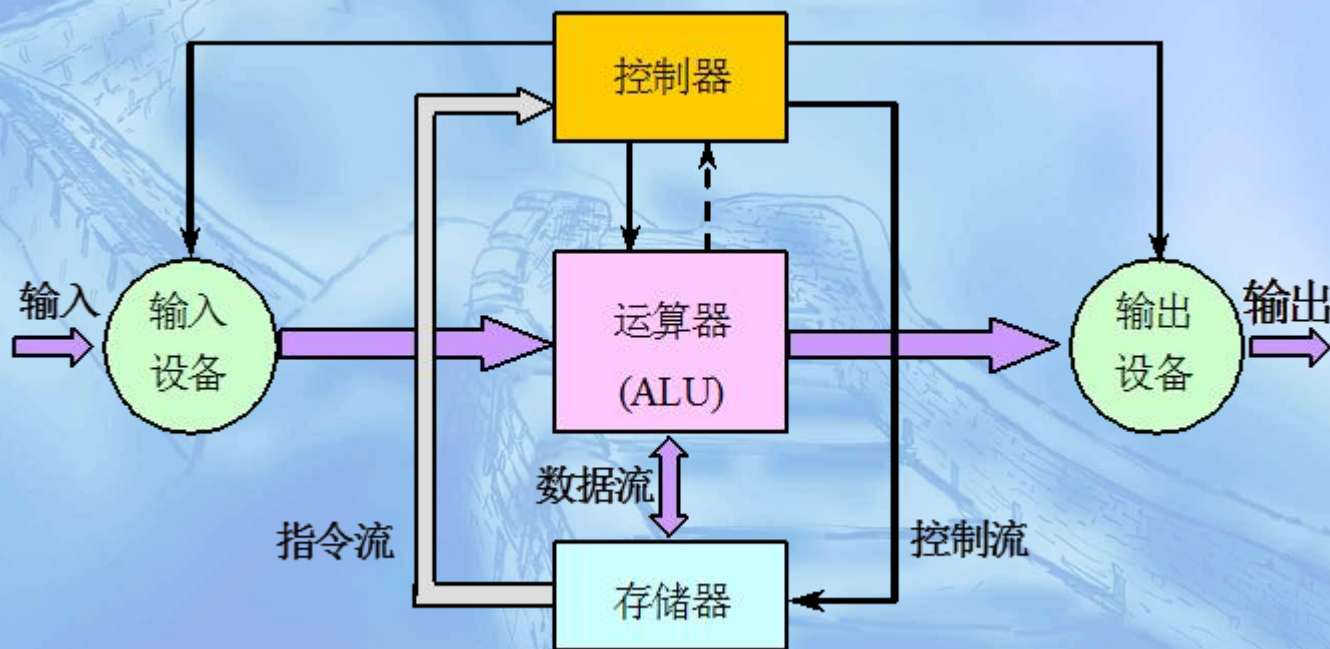
计算机系统结构 的发展

- ◆ 冯·诺依曼（Van Neumann）结构
- ◆ 器件发展对系统结构的影响
- ◆ 软件对系统结构发展的影响
- ◆ 应用对系统结构发展的影响



冯·诺依曼结构

基本思想于1936年~1946年期间形成，由冯·诺依曼等人于1946年提出。





冯·诺依曼结构

现在计算机系统结构已经在冯·诺依曼结构上得到不断的发展和改进，主要体现在下面两点：

- 合理地进行软硬件功能分配
- 提高并行处理能力



改进一

◆ 原来

以运算器为中心，所有部件由控制器集中控制。

◆ 改进

- 以主存为中心
- 操作并行

例如，I/O与CPU并行（DMA、I/O处理机等），I与O并行等。



改进二

◆ 原来

存储程序：指令顺序执行，指令和数据同等对待（存于同一存储器中，指令在执行中可修改）。

◆ 改进

➢ 数据流机器

指令的执行次序与存放次序没有关系，由数据驱动。

➢ 指令在执行过程中不准修改

➢ 指令和数据分开存储



改进三

◆ 原来

存储器按地址访问，是顺序、一维的线性空间，且每个单位的位数是固定的。

◆ 改进

- 采用按内容访问的相联存储器硬件实现高速相联查找
- 将整个访存地址分为段号、页号和页内地址
- 存储器同时具有按字、字节、位的多种编址方式



改进四

◆ 原来

指令在存储器中基本是按其执行顺序依次存储。

◆ 改进

- 先行控制、重叠和流水等（**CPU**内的指令执行）
- 向量处理机、并行处理机和多处理机等（指令内、指令间、任务间的并行）
- 数据流计算机（指令的执行由数据驱动，而非通常意义下的顺序执行）



改进五

◆ 原来

指令由操作码和操作数两部分组成，操作数的类型由操作码指明。

◆ 改进

- 增加许多高级数据表示，例如：自定义数据表示（数据自带类型标志）
- **CISC**（复杂指令系统计算机）、**RISC**（精简指令系统计算机）和**WISC**（可写指令系统计算机）



改进六

◆ 原来

数据以二进制编码，并采用二进制运算。

◆ 改进

- 增加基本数据表示

例如，十进制数、逻辑数、可变长字符串等。

- 增加高级数据表示

例如，自定义、向量、堆栈等。



器件发展对系统结构的影响

- ◆ 器件发展的特点
- ◆ 器件种类
- ◆ 器件发展对设计方法的影响



器件发展的特点

➤ 集成度迅速提高

目前水平：每0.25平方英寸 10^8 个晶体管，每5年左右提高一个数量级。

➤ 速度已接近极限

将来速度的提高主要靠系统结构、组成和实现技术。

➤ 价格直线下降

例如，CPU芯片刚推出时\$1000，一年后降到\$200。

➤ 可靠性越来越高

芯片可靠性到达 10^8 小时，即连续使用1年以上。



器件种类

◆ 通用片

用户不能改变其功能，在生产时定死，可以大批量生产，例如：**CPU**、存储器、寄存器、逻辑门...

◆ 现场片

用户可以在现场更改内容，例如：**PROM**、**EPROM**...

◆ 用户片

按用户要求专门生产的高集成度的**VLSI**器件。

➢ 全用户片

完全按用户要求设计的用户片，例如：各种专用芯片。

➢ 半用户片

按通用片生产的用户片，例如：门阵列片等。



器件发展对设计方法的影响

➤ 计算机组成技术下移

原来只有在巨型机、大型机中采用的技术，已经用到微型机中，例如：超流水线、**Cache**等。

➤ 计算机设计方法上移

第1代计算机，几乎全部由硬件人员设计；第2、3、4代，由应用、软件、硬件和器件人员共同设计；将来：除器件设计之外，主要是软件设计。

➤ 逻辑设计方法改变

过去逻辑设计很重要，尽量节省每一个芯片，每一个门；目前的设计主要考虑：了解市场，尽量选择通用芯片、标准芯片；结构化设计，规整、易读、易诊断。



软件对系统结构的影响

用户对软件的要求

- ◆ 执行速度
- ◆ 开发周期
- ◆ 可移植性 (**portability**)

若一个程序可以不经修改或只需少量修改就能运于不同型号的计算机上，则称该程序是可移植的 (**portable**)。



实现软件可移植性

- ◆ 统一高级语言
- ◆ 采用系列机思想
- ◆ 模拟与仿真



统一高级语言

◆ 思想

设计出一种完全通用的高级语言，为所有的程序员所使用。

◆ 存在问题

- 不同的用途往往要求语言有不同的语法结构和语义结构
- 人们对语言的基本结构还没有透彻的和统一的认识
- 习惯势力的影响

◆ 例子

Java语言。



采用系列机思想

先确定好系统结构，软件设计者据此设计软件，硬件设计者则根据不同的性能要求，采用不同的组成和实现技术，提供不同档次的机器。

◆ 系列机

是指在一个厂家内生产的具有相同的系统结构，但具有不同组成和实现的一系列不同型号的机器。例如，**IBM PC系列、IBM370系列**等。

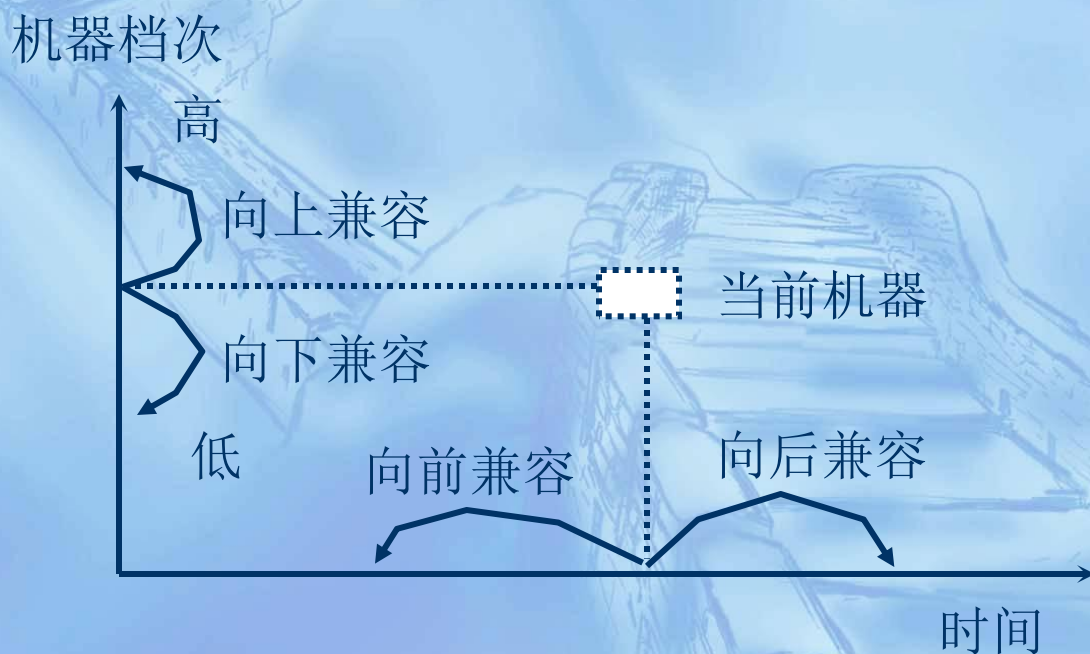
◆ 兼容机

是指不同公司厂家生产的具有相同系统结构，但具有不同组成和实现的一系列不同型号的机器。兼容机允许进行扩充，例如：**长城0520兼容机**增加了汉字处理功能



软件兼容性 (compatibility)

软件兼容性指的是同一软件可不加修改地运行于该系列机器中的各档机器上的特性。



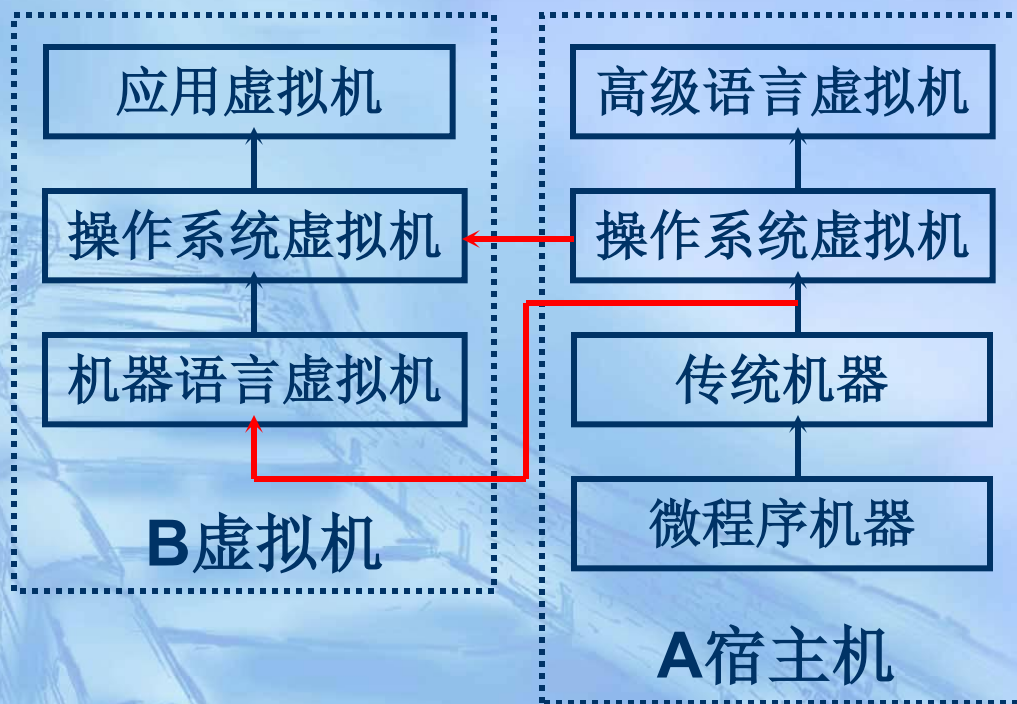
- **向后兼容**必须做到，而**向前兼容**却不一定。
- **向上兼容**一般要求做到，而**向下兼容**却不一定。



模 拟

模拟 (simulation) 指的是用一台机器 (**宿主机**) 的机器语言程序解释实现另一台机器 (**虚拟机**) 的指令系统的方法。

- 模拟可实现程序在任何机器间的移植
- 模拟影响虚拟机的速度
- 模拟不宜用于在时间关系上有约束的程序的移植

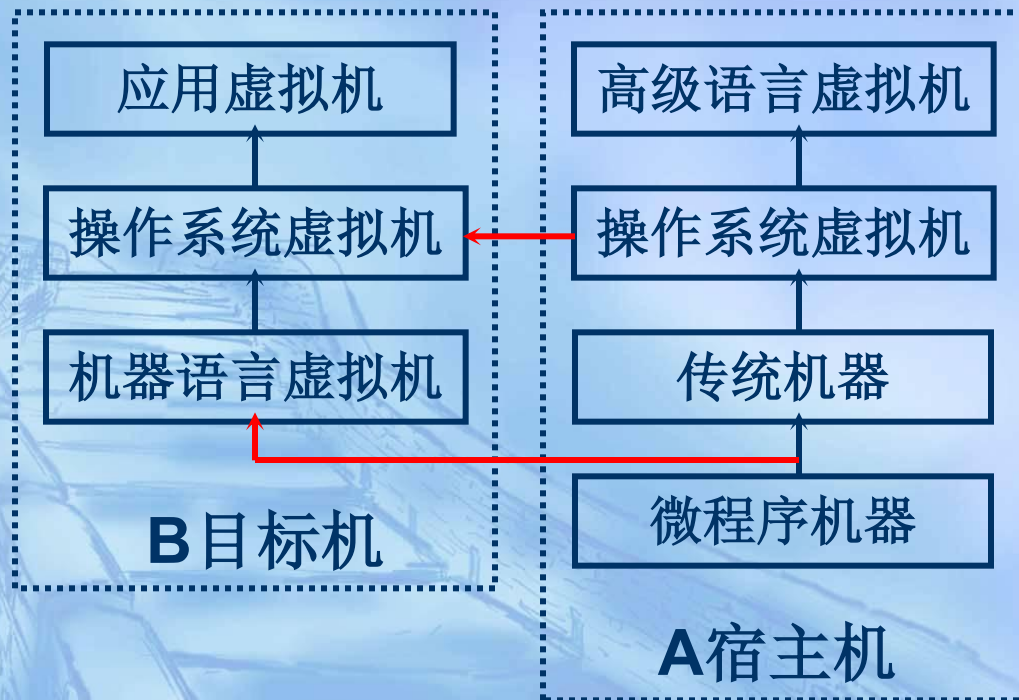




仿真

仿真 (emulation) 指的是用一台机器 (**宿主机**) 的微程序解释实现另一台机器 (**目标机**) 的指令系统的方法。

- 若目标机同宿主机的结构差别大，则仿真困难
- **模拟用机器语言**程序解释，其解释程序存在主存中。
- **仿真用微程序**解释，其解释程序存在控制存储器中。





模拟和仿真

在实际应用中，通常将模拟和仿真这两种方法混合使用。

- 频繁使用的指令→仿真
- 使用频度低和难以仿真的指令→模拟



应用领域的演变

不同的应用对计算机系统结构的设计提出了不同的要求，故**应用需求**是促使计算机系统结构发展的**最根本的动力**。

领域



时间



计算机的演变

专用机



通用机



高性能专用机



高性能通用机

设计的基本出发点是使**专用系统结构的高效率**与**通用系统结构的广泛市场**成均势。



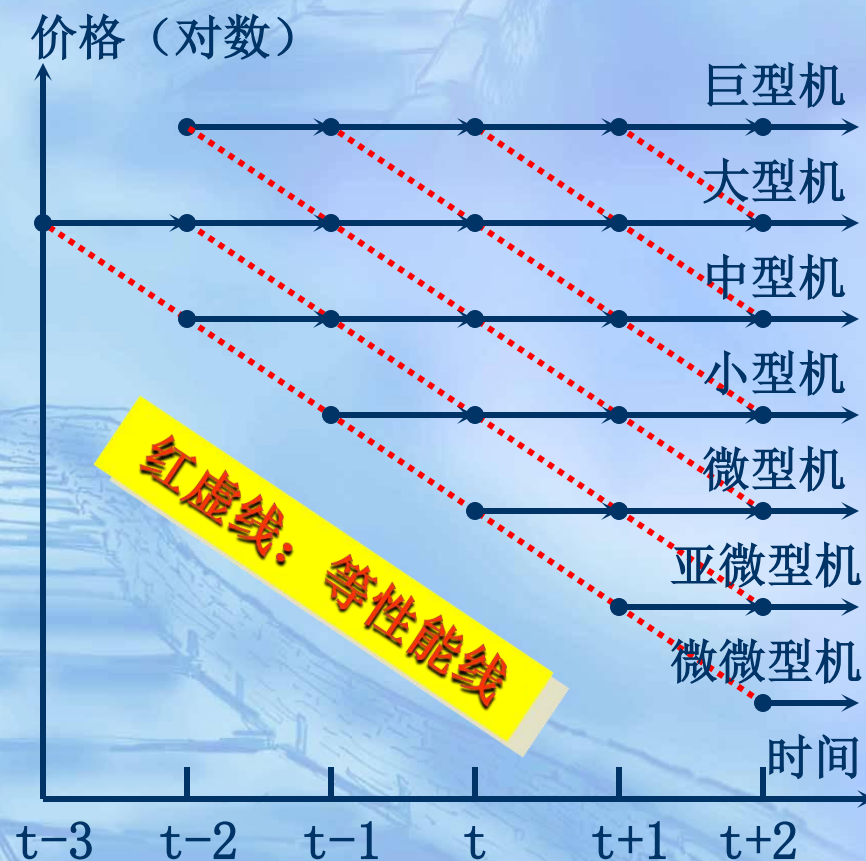
性能和价格关系的处理

◆ 问题

用户的性能/价格要求决定厂商的生产决策。

◆ 方法

- 维持价格不变，利用新型器件提高机器性能（水平实线）
- 保持性能基本不变，利用低成本器件降低机器价格（等性能线）





计算机发展方向

➤ 最高性能价格比

例如，商用机。这是计算机发展的主要方向。

➤ 最高性能

例如，银河系列、天河系列（国防科技大学计算机研究所），曙光系列（中科院计算技术研究所），神威系列（国家并行计算机工程技术中心）等，主要出于国家安全需要、科技发展需要。

➤ 最低价格

例如，家用学习机等。



计算机系统的分类

- ◆ 佛林分类法
- ◆ 库克分类法
- ◆ 冯泽云分类法



佛林(Flynn)分类法

按**指令流**和**数据流**的**多倍性**进行分类:

- ◆ 单指令流单数据流
- ◆ 单指令流多数据流
- ◆ 多指令流单数据流
- ◆ 多指令流多数据流

➤ 指令流

机器执行的指令序列。

➤ 数据流

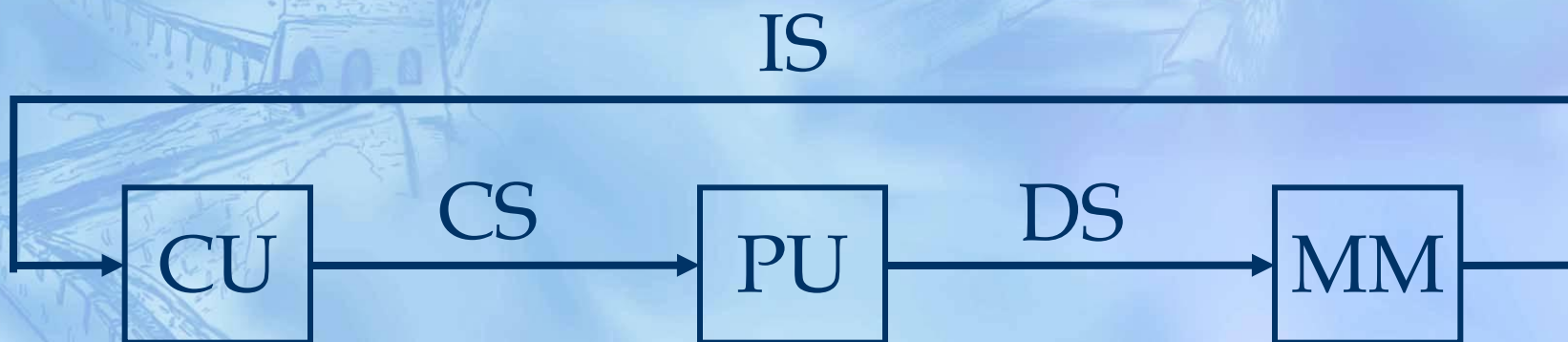
由指令流调用的数据序列（输入数据、中间结果）。

➤ 多倍性

在系统性能瓶颈部件上处于同一执行阶段的指令或数据的最大可能个数。



单指令流单数据流 (SISD)

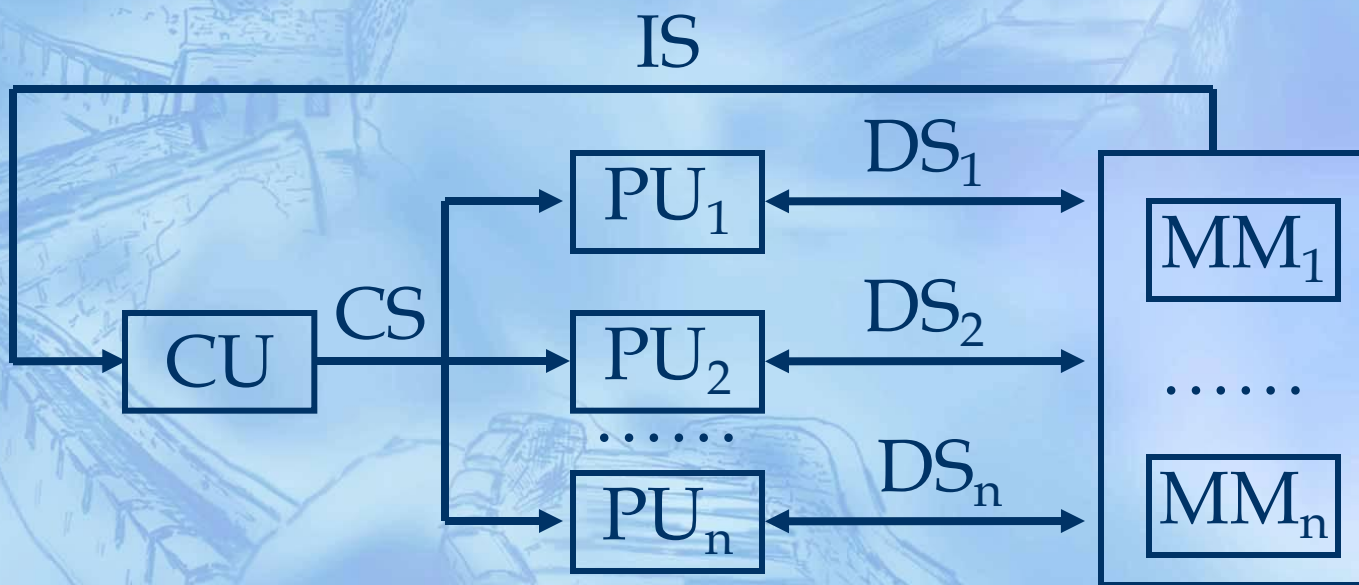


典型单处理机，包括：

- 单功能部件处理机：IBM1401，VAX-11
- 多功能部件处理机：IBM360/91，370/168，CDC6600
- 流水线处理机：标量流水线处理机



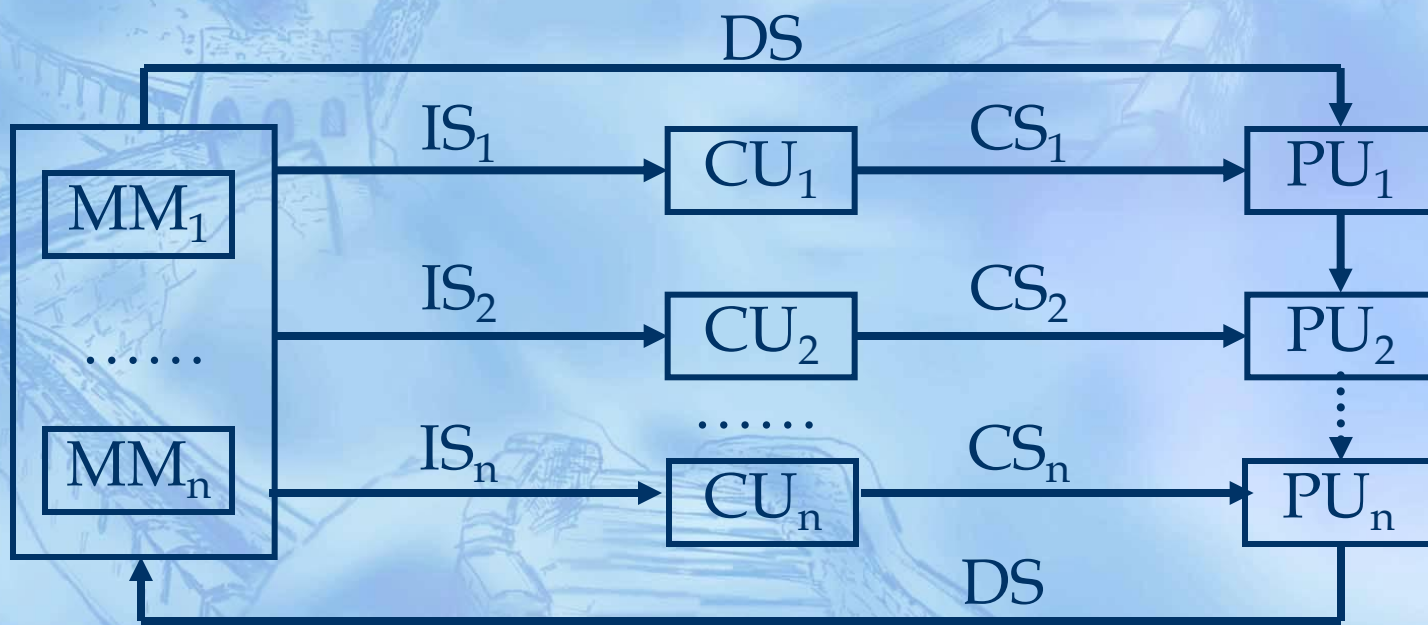
单指令流多数据流 (SIMD)



例如：并行处理机、阵列处理机、向量处理机、相联处理机、超标量处理机、超流水线处理机...



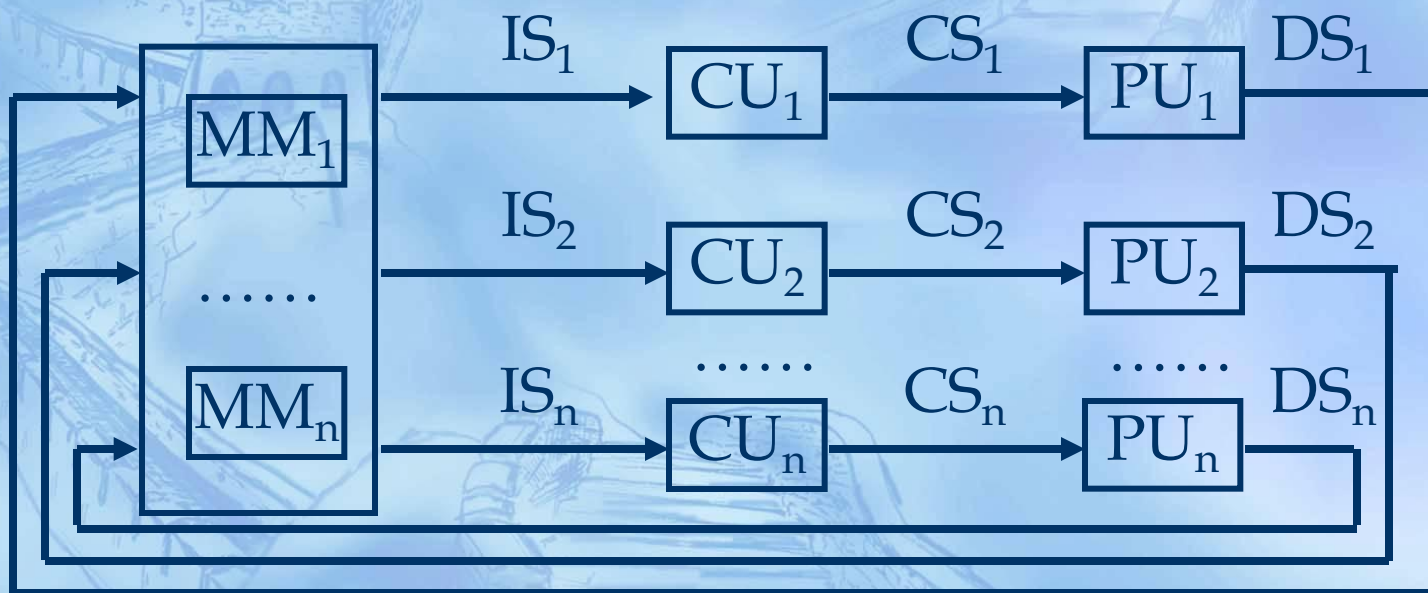
多指令流单数据流 (MISD)



几条指令对同一个数据进行不同处理，这种类型的计算机实际上不存在。



多指令流多数据流 (MIMD)



多处理机系统，包括，紧密耦合：UNIVAC-1100/80、IBM3084、IBM3081；松散耦合：D-825、CRAY-2、Cmmp。



特 点

Flynn分类法能反映出大多数计算机的并行性工作方式和结构特点，得到了广泛应用，但存在下列缺点：

- 分类太粗
- 只能对**Neumann**型机器分类
- 难以完全反映系统在工作原理上的差别
- 无法定量描述系统的类型



库克分类法

按指令流和执行流的多倍性进行分类:

- ◆ 单指令流单执行流 (SISE)

典型的单处理机。

- ◆ 单指令流多执行流 (SIME)

相联处理机、向量处理机、流水线处理机、超流水线处理机、超标量处理机、SIMD并行处理机。

- ◆ 多指令流单执行流 (MISE)

多道程序系统。

- ◆ 多指令流多执行流 (MIME)

典型的多处理机。



冯泽云分类法

用**最大并行度**来对计算机系统进行分类。

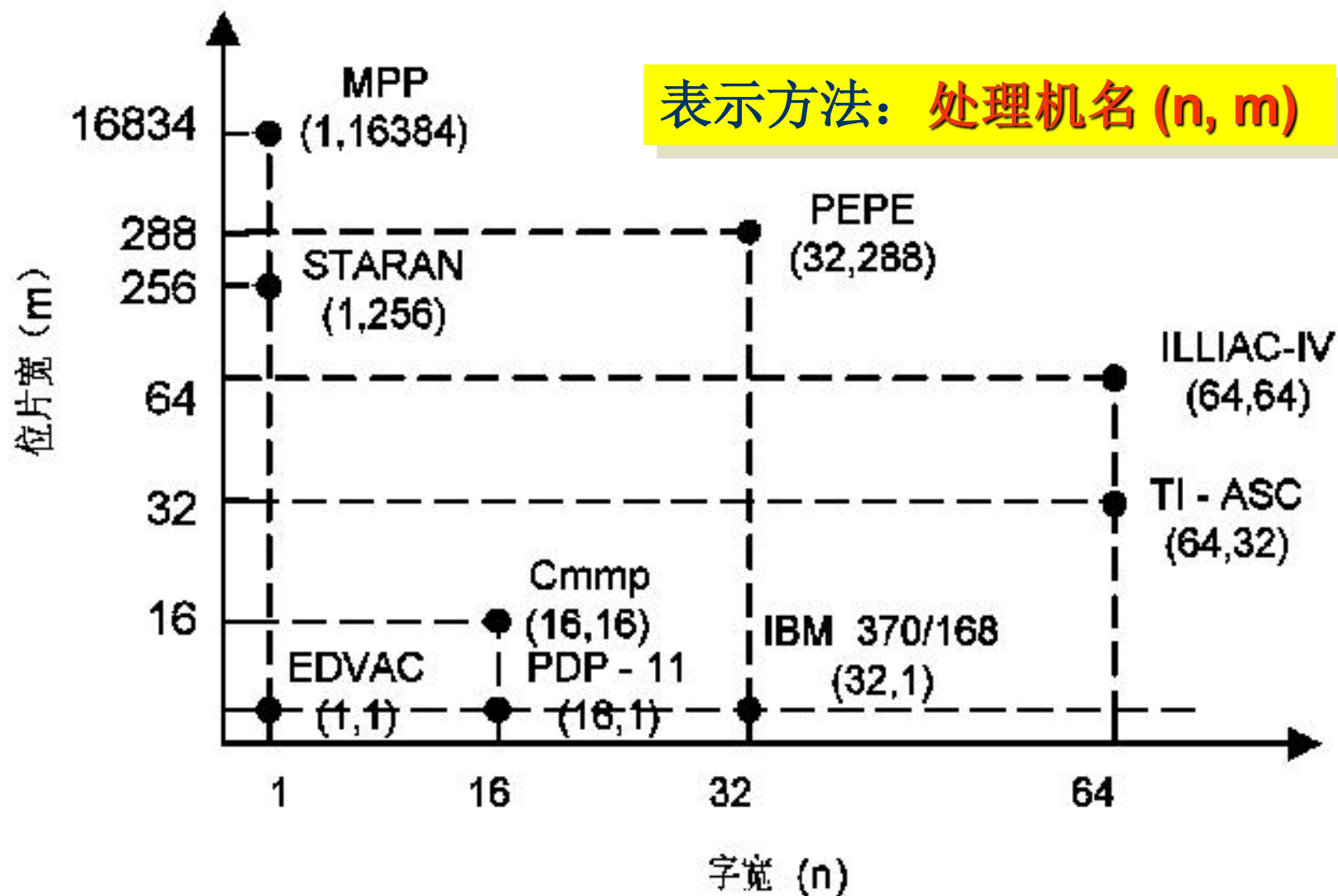
➤ 最大并行度

计算机系统在单位时间内能够处理的最大二进制位数。
假设同时处理的字宽为 n ，位片宽为 m ，则最大并行度定义为： $P_m = m \times n$ 。

➤ 平均并行度

假设每个时钟周期 t_i 内能同时处理的二进位数为 B_i ，则 n 个时钟周期内的平均并行度定义为：

$$P_n = \frac{\sum_{i=1}^n B_i \cdot t_i}{n}$$



按最大并行度的冯氏分类法





冯泽云分类法

用最大并行度来对计算机系统进行分类，可分为四种：

➤ **字串位串 (WSBS) : $n=1, m=1$**

串行计算机，例如：EDVAC(1, 1)。

➤ **字并位串 (WPBS) : $n>1, m=1$**

传统的单处理机，例如：Pentium(32,1)。

➤ **字串位并 (WSBP) : $n=1, m>1$**

并行计算机、MPP、相联计算机；例如：MPP(1, 16384)、STARAN(1, 256)、DAP等。

➤ **字并位并 (WPBP) : $n>1, m>1$**

全并行计算机，例：ILLIAC IV(64,64)、ASC(64,32)、PEPE、Cmmp等。

